

When Protocol States Mislead: Evidence-Gated Admission and Online Feedback Calibration for LLM-Assisted Stateful Fuzzing

Ketang Chen^a, Xiaolei Ren^{a,*}

^aMacau University of Science and Technology, Avenida Wai Long, Taipa, Macau, China

Abstract

Response-derived states offer inexpensive feedback for stateful protocol fuzzing, but state novelty is an imperfect proxy for code progress. Requests generated by large language models (LLMs) can amplify this mismatch when unproductive proposals receive durable mutation resources. We present LoopFuzz, an evidence-calibrated controller design combining execute-before-promote admission with online state-productivity calibration. Bounded trials separate structural validity, response acceptability, and reachability from code and inferred protocol state machine (IPSM) novelty. Code-supported candidates qualify for durable admission; state-only candidates receive provisional validation and expire without subsequent code evidence. Discounted Beta–Bernoulli updates and Thompson sampling combine estimated code productivity with frontier exploration, without claiming to recover semantic protocol states. An audit of 628 archived runs across nine targets provides descriptive coverage, cost, and mechanism measurements. Observed code-branch and IPSM state-edge endpoints do not consistently reproduce the hypothesized mismatch across motivating targets. Four vulnerability case studies add replay evidence; controlled rediscovery benefits remain unmeasured. A five-arm evaluation protocol specifies matched model configurations, intervention policies, and resource ceilings to isolate admission and calibration. Heterogeneous observation horizons and incomplete lifecycle evidence restrict current findings to descriptive results.

Keywords: Stateful protocol fuzzing, Large language models, Response-derived state feedback, Evidence-gated admission, Online productivity calibration

1. Introduction

Stateful protocol fuzzing must discover useful interaction sequences as well as useful message contents. A server can accept a request only after earlier exchanges establish a session, authenticate a client, or create a resource. AFLNet uses responses to construct an inferred protocol state machine (IPSM), making protocol-visible feedback available without a target-specific internal-state extractor [1]. This deployment advantage comes with an abstraction boundary: a response code or sequence is evidence about observable behavior, rather than an identifier of the server’s complete semantic state.

StateAFL already identifies limitations of response-derived state abstractions and uses memory observations to infer server states [2]. The fact that responses can misrepresent internal progress is therefore established background. Our question is how a fuzzer retaining a response-visible abstraction should decide, during a campaign, whether a particular state signal is useful enough to justify further exploration. A new IPSM state edge can accompany a previously unseen code branch, reproduce an already explored rejection path, or distinguish responses after the measured code surface has saturated. These outcomes should not receive identical evidential weight.

Assistance from large language models (LLMs) makes this distinction consequential for queue management. ChatAFL demonstrates that LLMs can derive protocol structures and propose interaction sequences [3]. A generated request is nevertheless a hypothesis about both message structure and live session conditions. If a proposal is retained indefinitely because it produces a new response-derived state, the fuzzer may repeatedly mutate an input whose apparent novelty has little downstream code value. Durable queue membership represents a continuing allocation of execution resources.

Three target families motivate the investigation. Forked-daapd exposes DAAP/DACP behavior through HTTP status responses, so response distinctions may overstate useful code progress. Lighttpd1 can execute different code paths under the same HTTP status, so response novelty may understate code progress. LightFTP motivates examining continuing IPSM growth after code-coverage saturation. These are hypotheses inherited from earlier development observations, not conclusions imposed on the present archive. Section 2 separates them from the available measurements.

LoopFuzz addresses this problem through evidence-gated admission and online state-productivity calibration. It first checks a proposal’s structure, then executes a bounded trial and separates response acceptability, state reachability, code novelty, and IPSM novelty. Code-supported candidates can enter the durable queue. State-only candidates receive a provisional validation budget and must earn durable promotion through subsequent

*Corresponding author: Xiaolei Ren.

Email addresses: 3250006913@student.must.edu.mo (Ketang Chen), x1ren@must.edu.mo (Xiaolei Ren)

code evidence. At the scheduling layer, a discounted productivity estimate conditions investment on the observed relationship between selected response states and later code progress.

This work makes three contributions, with their current evidence status stated explicitly.

1. **Evidence-gated queue admission.** We specify an execute-before-promote controller that distinguishes reject, provisional, and durable outcomes. It separates state novelty from code evidence and makes the lifetime and descendant cost of provisional entries auditable.
2. **Online calibration of state productivity.** We formulate response-state utility as the probability of code progress in a bounded state-selection episode. Discounted Beta-Bernoulli updates and Thompson sampling instantiate the controller; these established estimation and selection methods are not claimed as new algorithms.
3. **A controlled empirical framework with partial evidence.** We define candidate-level and episode-level measurements, five arms, launch accounting, and information-controlled rediscovery on vulnerable/patch-paired targets. We audit the supplied archive, report four vulnerability replay dossiers with their evidence limits, and identify the missing evidence for causal admission and calibration claims.

The available evidence does not justify describing the third contribution as a completed five-arm evaluation. Preserving this distinction allows the design, implementation, and later experiments to be evaluated against the same explicit contract.

2. Background and Motivation

2.1. Response-visible state and independent code feedback

Let $\mathcal{M}_t = (Q_t, \widehat{\Delta}_t, q_0)$ be the IPSM observed by time t , where Q_t contains response-derived state identifiers, $\widehat{\Delta}_t \subseteq Q_t \times Q_t$ contains observed IPSM state edges, and q_0 denotes the initial connection state. The abstraction can alias different internal states and separate executions that reach the same code. It can also depend on history, asynchronous replies, and reset behavior. It is not assumed deterministic or semantically complete.

Let $\mathcal{B}_t$ be the covered code-branch set under a fixed instrumentation and replay pipeline. An IPSM state-edge increment changes $\widehat{\Delta}_t$; a code-branch increment changes $\mathcal{B}_t$. These objects have distinct units. An AFL bitmap observation must also distinguish a newly encountered code edge from a hit-count bucket change. Neither bitmap slots nor response-state identifiers are interchangeable with source-level code branches.

2.2. Three falsifiable mismatch cases

Table 1 defines the three motivating cases. Figure 1 summarizes the archived A/D endpoints as mean $\pm$ sample standard deviation (SD), with actual run counts n and nominal $N = 10$. Establishing a mismatch mechanism requires comparable builds, a shared observation horizon, and execution-level or time-resolved evidence. Endpoint differences alone do not establish which

state caused a difference. The partial archive must be allowed to contradict the earlier negative-target narrative.

The desired controller must accommodate both overestimation and underestimation. Rejecting every state-only candidate could lose useful bridges; promoting every such candidate would treat an imperfect proxy as sufficient evidence for permanent resource allocation. A provisional queue expresses the intermediate conclusion that an input deserves limited descendant validation while its longer-term value remains unproven.

The scope is textual or semi-textual request-response protocols with usable parsers, response extractors, and code feedback. Response-visible state extraction avoids a target-specific internal-state adapter, but the complete controller remains grey-box because productivity requires code feedback. It is not a pure black-box system. Encrypted or opaque binary protocols require suitable construction, response, and code-observation adapters before evaluation.

3. Related Work and Novelty Boundary

Stateful greybox fuzzing studies how to expose, infer, and schedule protocol state [1, 2, 4, 5]. AFLNet supplies a response-derived IPSM, whereas StateAFL and NSFuzz use different internal observations. LoopFuzz retains the response-visible interface and studies its instrumental reliability under an independent code-progress criterion. It does not claim to be the first system to identify state aliasing or misleading responses.

LLM-assisted generation addresses structured inputs and interaction histories [3, 6, 7]. Grammar-aware and documentation-guided methods likewise improve input structure [8, 9]. Better proposals still need execution evidence before acquiring long-term queue resources. Our controlled ChatAFL comparison preserves its scheduling lineage rather than replacing its scheduler.

State and seed scheduling already use adaptive resource allocation. The Bandit’s States studies bandit-based protocol-state selection [10]; Zhang et al. study seed scheduling with Thompson sampling [11]; SSGFuzz studies state significance in protocol fuzzing [12]. These systems delimit the novelty claim: a bandit or productivity score is not itself a contribution. The distinction here is the combination of a response-derived proxy, an independent code-progress target, two-tier LLM-candidate admission, and auditable episode evidence.

Benchmark methodology requires repeated trials, comparable instrumentation, explicit budgets, and careful failure interpretation [13–15]. Historical vulnerability benchmarks such as Magma [16] motivate distinguishing execution reaching bug-related code from execution satisfying a trigger condition. We adopt that distinction for network-reachable vulnerable/patch pairs; crash-file counts are not the unit of security success.

4. Evidence-Calibrated Controller

This section specifies the controller contract. Section 5 identifies where the inspected implementation and archive do not yet satisfy it. A mathematical design rule is not evidence that a completed campaign obeyed that rule.

Table 1: Motivating hypotheses and current evidence.

Target	Hypothesis	Required observation	Current interpretation
Forked-daapd	Overestimation: IPSM growth without downstream code gain.	Growing IPSM structure with low episode code reward.	Prior endpoint code regression not reproduced.
Lighttpd1	Underestimation: code progress within shallow response states.	Code-branch gain with little IPSM gain, linked to executions.	Both endpoints increase; hidden productivity remains unverified.
LightFTP	Saturation: IPSM novelty after code progress plateaus.	Aligned code/IPSM trajectories across saturation.	Near-equal code endpoints; current-build trajectories still needed.

Descriptive native endpoints

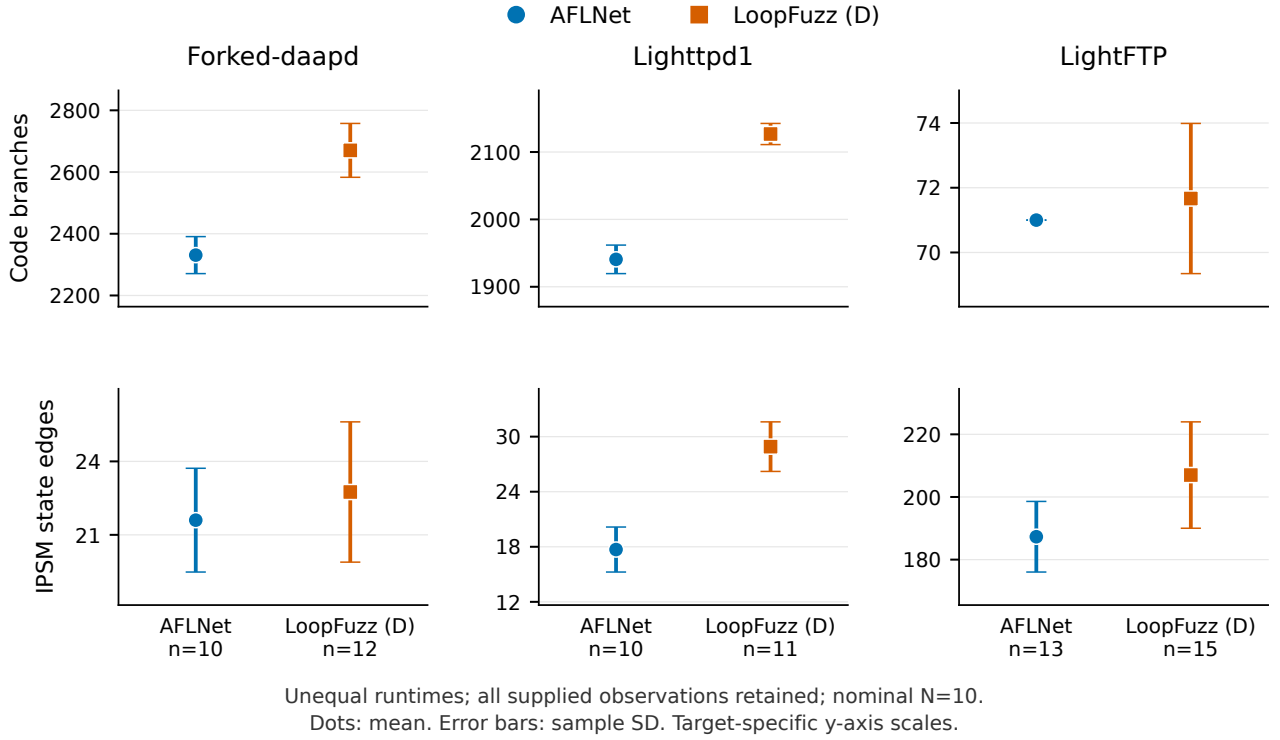
Figure 1: Native A/D endpoints (mean $\pm$ SD; actual n shown).

Table 2: Prior mechanisms and novelty boundaries.

Prior work	Established focus	Present boundary
AFLNet; StateAFL; NSFuzz	Response-derived or internal-state feedback.	Calibrate the response proxy; no semantic-state recovery.
ChatAFL; LLM synthesis	Protocol-informed message/sequence generation.	Execute-before-promote; bounded provisional validation.
The Bandit’s States [10]	Bandit-based protocol-state selection.	Prior bandit formulation; independent reward and queue evidence here.
Zhang et al. [11] SSGFuzz [12]	Thompson-sampling seed scheduling. State-significance-guided fuzzing.	Reused Beta estimation and Thompson sampling. Code-productivity calibration, distinct from state importance.
Magma [16]; evaluation methodology	Ground-truth bug conditions; repeated evaluation.	Information-controlled network rediscovery; minimal security-patch pairs.

4.1. LLM proposal interface

The model receives permitted RFC fragments, seed traces, and observed interaction summaries and returns candidate requests or bounded scheduling hints. Each candidate has a per-

sistent identifier, source state, intended target/frontier, and links to its prompt and raw reply. Message-field hypotheses remain proposals rather than verified protocol grammars.

The information policy excludes CVE identifiers, security

Table 3: Admission predicates and required evidence.

Predicate	Positive evidence	Example failure	Required audit fields
P	Schema and structural checks pass.	Missing field; invalid length/delimiter.	Candidate ID; parser/schema version; reason; hypothesis hash.
U	Adapter accepts response.	Rejection; timeout/reset/drop; unparseable reply.	Response code/hash; adapter; flags; exception rule.
R	Target/frontier reached or documented sink escape.	Repeated sink; missed destination.	Source/target; observed state sequence.
G_{code}	New code branch/code edge or code-supported favored entry.	IPSM-only novelty.	Pre/post code coverage; novelty class; favored reason.
G_{state}	New IPSM node, state edge, or path.	Known IPSM structure.	Pre/post IPSM; new transitions.

advisories, exploit inputs, and patch contents. It fixes which documentation and live responses may be supplied. Identical information policy does not imply identical prompt bytes after adaptive campaigns diverge; prompt and reply hashes expose that distinction.

Outputs that propose requests pass through admission. Outputs that only guide scheduling require schema, state-ID, and seed-ID checks and cannot insert durable inputs. Parsing or transport failure does not create a positive admission decision. All attempts, retries, failures, and successful replies enter the declared call/token accounting.

4.2. Admission predicates and evidence separation

For candidate x , $P(x)$ checks schema, message structure, required fields, lengths, and terminators before execution. A bounded trial then evaluates response acceptability $U(x)$ and reachability $R(x)$: arrival at a declared target/frontier or escape from a recorded rejection sink. Table 3 gives operational evidence and required reason codes.

HTTP/FTP 4xx/5xx classes are rejection priors, not a universal semantic definition. An adapter must specify exceptions before a run and retain the response and reason code. Timeout, reset, disconnect, or an unparseable response gives no positive U evidence. A valid response alone is insufficient for R . Security-oracle artifacts are retained independently even when normal request admission fails.

Gain is split into

$$G_{\text{code}}(x) = \mathbf{1}\{\text{new code coverage or a favored code-coverage entry}\}, \quad (1)$$

$$G_{\text{state}}(x) = \mathbf{1}\{\text{new IPSM node, state edge, or path}\}. \quad (2)$$

Favored status must be grounded in code coverage; merely being queued or state-novel does not qualify. Source-level code branches are the main evaluation surface. If runtime instrumentation observes code edges instead, that reward definition must be frozen and reported separately. Hit-count novelty is a separate sensitivity condition. Pre-trial coverage and IPSM snapshots establish which gains are new.

4.3. Provisional and durable queues

Let Q_{prov} contain temporary candidates and Q_{dur} contain long-term mutation seeds. Admission has three outcomes:

$$D(x) = \begin{cases} \text{durable,} & P \wedge U \wedge R \wedge G_{\text{code}}, \\ \text{provisional,} & P \wedge U \wedge R \wedge \neg G_{\text{code}} \wedge G_{\text{state}}, \\ \text{reject,} & \text{otherwise.} \end{cases} \quad (3)$$

IPSM novelty alone is never sufficient for durable promotion. These logical conditions do not assume statistical independence among predicates.

A provisional entry receives at most $H = 64$ descendant executions and a local validation deadline of 30 seconds, whichever is exhausted first; the proposed live provisional capacity is 64. These initial settings are checked in the pilot and frozen before formal runs. Budget checks occur before each descendant execution. Exhaustion expires the entry from active scheduling while preserving its audit record. Campaign termination before the validation horizon is censoring, not unproductive expiration.

A descendant supplying qualifying code evidence permits promotion of the replayable parent sequence and productive descendant, subject to the same structural, response, and reachability checks. The record identifies the first productive descendant and the candidate whose status changes. A global code-discovery event is credited once through execution/lineage IDs, not repeatedly to multiple ancestors.

Productivity estimates select where to propose and continue exploration; they do not independently relax Equation 3. Fixed and calibrated arms use identical admission rules and provisional budgets. Calibration thus affects admission outcomes indirectly through exploration opportunities, preserving the causal scheduling comparison.

4.4. State-selection episodes and independent reward

An episode begins when the scheduler selects response-derived state s and a seed capable of reaching it. It receives one fixed execution-energy budget. Individual executions within the episode do not generate repeated posterior updates. Mutation counts, reset failures, elapsed time, and budget exhaustion are logged. Infrastructure-interrupted episodes are incomplete, rather than silently assigned reward zero.

For completed episode t targeting s_t , let ΔB_t count previously unseen code branches discovered within that episode. The main reward is

$$r_t = \mathbf{1}\{\Delta B_t > 0\}. \quad (4)$$

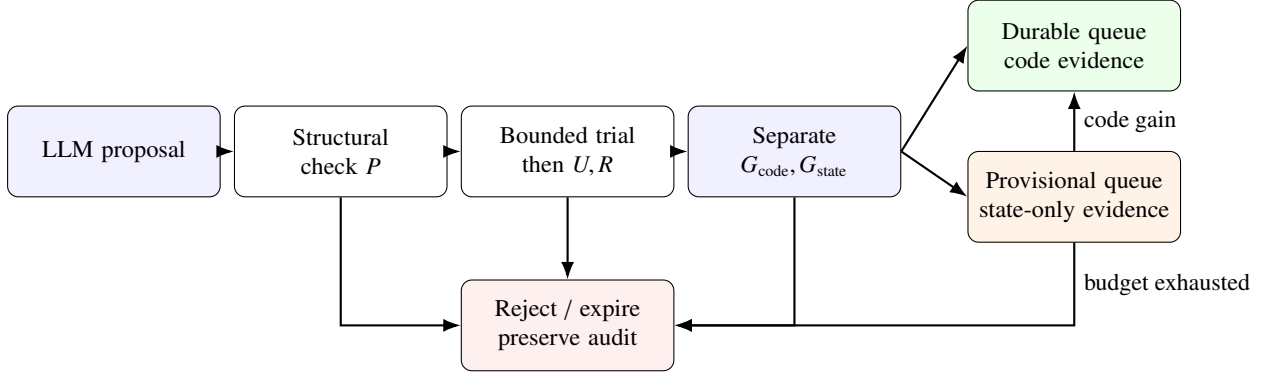


Figure 2: Evidence-gated admission with provisional and durable queues.

A declared code-edge instrumentation variant is admissible, but IPSM novelty is excluded. Favored descendants are a secondary sensitivity reward. Equal energy prevents a state appearing more productive merely because it receives more executions. Trial execution, ordinary mutation, and provisional validation use distinct event types to avoid duplicate updates.

θ_s denotes the probability of positive code reward conditional on selecting s under the episode policy and current campaign phase. It is instrumental utility, not the probability that s is a true semantic state. U continues to mean response acceptability; utility is never denoted $U(s)$.

4.5. Discounted productivity estimates

Each state starts with $\alpha_s = \beta_s = 1$ and

$$\theta_s \sim \text{Beta}(\alpha_s, \beta_s), \quad \widehat{\theta}_{s,t}^{\text{pre}} = \frac{\alpha_{s,t}}{\alpha_{s,t} + \beta_{s,t}}. \quad (5)$$

For a completed selected-state episode:

$$\alpha_s \leftarrow 1 + \gamma(\alpha_s - 1) + r_t, \quad (6)$$

$$\beta_s \leftarrow 1 + \gamma(\beta_s - 1) + (1 - r_t). \quad (7)$$

The proposed discount is $\gamma = 0.995$, with sensitivity values $\{0.99, 0.995, 1.0\}$. Parameters are frozen after the pilot and before inspecting formal results. At $\gamma = 1$, updates reduce to cumulative Beta–Bernoulli counts. For $\gamma < 1$, excess prior counts are discounted to respond to changing productivity as coverage saturates.

The term productivity posterior denotes this discounted working model. With discounting it is not the exact posterior of a stationary Bernoulli process. Discounting occurs on completed episodes for the selected state; unselected states do not automatically age in wall-clock time. The update clock and cold start are therefore explicit applicability limits.

Every calibration metric uses the pre-update mean recorded before reward. Post-update predictions would leak the outcome into their own evaluation. Thompson samples are recorded separately and are not substituted for predicted Bernoulli probabilities in reliability calculations.

4.6. Calibrated scheduling

The topology term favors underexplored observed states:

$$F(s) = \begin{cases} 4, & s \text{ is newly reachable or } \deg^+(s) \leq 1, \\ 2, & 1 < \deg^+(s) \leq 3, \\ 1, & \deg^+(s) > 3. \end{cases} \quad (8)$$

Only reachable states with eligible seeds participate. $\deg^+(s)$ counts outgoing IPSM state edges, not code branches. Let $B(s)$ denote the inherited base weight shared by C, D, and E. Define

$$\text{Frontier}(s) = B(s)F(s). \quad (9)$$

Keeping this factor and any warm-up policy fixed prevents calibration from simultaneously changing the baseline scheduler. Using the Thompson-sampling principle [17], draw

$$\widetilde{\theta}_s \sim \text{Beta}(\alpha_s, \beta_s) \quad (10)$$

and calculate

$$S(s) = \text{Frontier}(s)[\epsilon + (1 - \epsilon)\widetilde{\theta}_s], \quad \epsilon = 0.1. \quad (11)$$

Select a state with probability proportional to $S(s)$, recording the pseudorandom seed. This is Thompson-sampled weighted selection, not an argmax policy. Under a finite eligible set, the positive floor gives nonzero selection probability, not a fixed number of selections.

The fixed-policy comparator retains the original penalty:

$$S_D(s) = \text{Frontier}(s)\rho_{\text{fixed}}(s). \quad (12)$$

Its cached queue-yield ratio $p_s = D_s/(N_s + 1)$ counts interesting queued paths D_s per state selections N_s . It is not a calibrated Bernoulli probability and can reflect state-only novelty or unequal execution energy. E replaces only ρ_{fixed} with the calibrated factor in Equation 11, preserving the shared base, admission, prompts, triggers, repair settings, and caps. The source implementation’s integer score conversion and initial round-robin cycles require explicit freezing and verification before comparison.

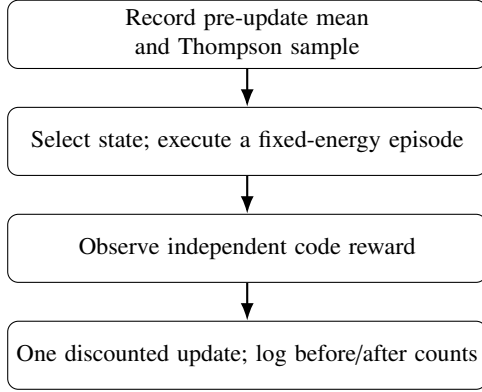


Figure 3: Online productivity calibration using pre-update predictions and code rewards.

Algorithm 1: Evidence-gated candidate processing

Input: Candidate x , target, immutable pre-trial snapshot
 1 Record candidate, prompt, reply, and lineage identifiers
 2 **if** $\neg P(x)$ **then**
 3 Record structural rejection; stop processing x
 4 Execute one bounded trial without durable queue insertion
 5 Record $U, R, G_{\text{code}}, G_{\text{state}}$ and reason codes
 6 **if** $P \wedge U \wedge R \wedge G_{\text{code}}$ **then**
 7 Promote x to Q_{dur}
 8 **else if** $P \wedge U \wedge R \wedge \neg G_{\text{code}} \wedge G_{\text{state}}$ **then**
 9 Insert x into Q_{prov}
 10 **while** *budget, validation deadline, and campaign time remain* **do**
 11 Check budget before the next descendant execution
 12 Execute and record one descendant
 13 **if** *qualifying code gain and validation pass* **then**
 14 Record first productive descendant; promote; end validation
 15 Record expiration or campaign censoring if still provisional
 16 **else**
 17 Record rejection
 18 Preserve security-oracle evidence separately

4.7. Intervention policy and optional repair

Plateau assistance uses a frozen definition, cadence, counters, and retry policy. The legacy trigger combines IPSM state-edge growth with an unproductive streak. If retained, its dependence on the proxy remains common to the causal arms and is reported; calibrating scheduling does not automatically calibrate triggering.

Primary comparisons of C, D, and E disable repair. Local repair guided by counterexamples remains an optional implementation feature evaluated diagnostically. E-with-repair versus E-without-repair records failure type, counterexample, parent repair ID, before/after hypothesis hashes, changed fields, validation, post-repair predicates, descendant gain, and token/latency cost. Sparse events or illustrative examples do not establish an effect. An additional contribution would require sufficient events, reproducible code gains, and independent statistical sup-

port across several targets.

5. Implementation and Evidence Status

The source includes split gain fields, provisional-queue machinery, and a posterior-scheduling switch. Their presence does not establish that archived runs exercised or correctly enforced the intended mechanisms. Run configuration, execution ordering, and event semantics must be checked together.

Several differences are material. The candidate path invokes native execution before evaluating the post-trial gate; native execution can already save a code-novel input. A later U/R failure does not undo that insertion, so the inspected implementation does not strictly enforce Equation 3. Native bitmap novelty includes hit-count changes, rather than exclusively new code branches. Provisional accounting checks budgets after a complete fuzzing round and can overshoot the nominal limits.

In addition, trial IPSM novelty is computed from graph updates tied to queue saving. In the inspected ordinary gated path, an unsaved candidate does not independently update that graph before the state-only check, whereas a native save already supplies code evidence. This ordering obstructs the intended state-only provisional branch. Zero provisional records therefore cannot be interpreted as successful filtering or proof that provisional validation is unnecessary.

Existing P flags can reflect action-schema acceptance rather than complete wire-format validation. The R implementation also admits generic parsed transitions without requiring the intended target/frontier. These are weaker conditions than the design definitions. Table 4 summarizes the boundaries. This manuscript revision does not change fuzzer code; it separates the proposed contract from the implementation evidence.

5.1. Append-only event contract

Six core streams record run configuration, candidate generation, admission trials, provisional lifecycle, state-selection episodes, and bug-oracle events. Repair adds a seventh optional stream. Table 5 specifies joinable fields. Every event also records the run ID, schema version, sequence number, and timestamp. CVE identities remain isolated benchmark metadata. Missing fields are not replaced with zero.

Before formal runs, candidate-event completeness must reach at least 99.9%; no trial may lack a candidate ID, no posterior update may lack an episode ID, and every promotion must be reconstructable. Completeness denominators include failed generation and parsing paths, not only successful candidates. File presence, join completeness, and agreement between logged disposition and actual queue state are separate checks.

6. Experimental Protocol

This protocol is prospective for unfinished confirmatory experiments. The supplied archive does not establish a dated preregistration, parameter freeze, or completed pilot. We do not retrospectively call it preregistered. A version, timestamp, and

Table 4: Controller requirements and implementation gaps.

Contract	Current boundary	Evaluation consequence
Gate precedes durable promotion	Native saving may precede U/R checks.	Disposition labels do not verify queue protection.
Independent state-only trial evidence	IPSM updates depend on native saving.	Zero provisional events do not establish a low state-only rate.
Full structural and reachability checks	P : action schema; R : generic transitions.	Audit parser checks and target reachability.
Main reward is new code coverage	Bitmap novelty includes hit counts; named branches may count slots.	Separate source branches, bitmap slots, and save events.
Fixed-energy episode	Variable mutation energy; zero-execution attempts.	Raw episodes do not provide equal opportunities.
Strict provisional budget	Limits checked after whole fuzzing rounds.	64-mutation/30-second limits may be exceeded.
Verified run configuration	Start event precedes some environment-flag parsing.	Cross-check launcher, end records, and execution path.
Equal total call/token caps	64-call cap covers plateau use; token enforcement unverified.	Total-budget fairness remains unverified.
Repair disabled in C/D/E	Defaults/activation differ from the required configuration.	Archive labels do not certify the D-arm contract.

Table 5: Required event fields by function.

Stream	Context / provenance	Measurements / outcomes
run_config	Arm; target/fuzzer commits; image digest; model/version/endpoint; temperature, top- p , penalties; seed.	Call/token caps; CPU/memory quotas; start/end times; termination reason.
candidate_event	Candidate/prompt/parent IDs; prompt/raw-reply hashes; raw input or artifact path; action; source/requested states.	Input/output tokens; generation timestamp.
admission_trial	Candidate/trial IDs; $P/U/R$ and reasons; response code/hash; timeout/reset/drop; state sequence; target/frontier reached.	Pre/post IPSM nodes/state edges and code coverage; favored reason; latency; disposition.
provisional_event	Stable candidate/provisional IDs; budget; first productive descendant.	Actual descendant executions/code gain; conversion time; promotion latency; expiration/censoring reason.
state_selection_episode	Episode/state/seed IDs; pre-update $\alpha, \beta, \hat{\theta}$; Thompson sample; frontier/final scores.	Mutations; code-branch/IPSM state-edge gain; reward; post-update α, β ; latency; completion status.
bug_event	Anonymous oracle ID; reproducing input; sanitizer evidence; root-cause signature.	Reached/triggered; first trigger time; vulnerable/patched results.
repair_event (optional)	Repair/parent IDs; failure/counterexample; before/after hypothesis hashes; changed field.	Before/after validation; post-repair predicates/descendant gain; tokens/latency.

checksum must be frozen before the remaining runs, with subsequent deviations recorded. Development and pilot observations remain separate.

6.1. Research questions

1. **RQ1: State/code alignment.** How reliably does response-derived state novelty predict downstream code progress across targets and campaign phases? Measure IPSM state-edge/code-branch alignment, episode productivity, calibration, and boundary cases.
2. **RQ2: Admission.** Does execute-before-promote reduce durable queue pollution and improve downstream productivity under matched proposal and scheduling conditions? Compare D with C.
3. **RQ3: Calibration.** Does online productivity calibration improve scheduling over a fixed response-state heuristic? Compare E with D.
4. **RQ4: Historical vulnerability rediscovery.** Does the controller improve reproducible rediscovery under independent reached/triggered oracles and patch validation?

5. **RQ5: Cost and applicability.** What runtime, throughput, and LLM costs arise, and under which target characteristics does the method fail to help?

The directional hypotheses are improved downstream productivity for D over C and greater area under the code-branch coverage curve (AUC) for E over D. Regressions, null results, and uncertainty are reported per target rather than hidden in pooled means or replaced by increased IPSM state-edge counts.

6.2. Five arms and causal comparisons

C/D share model configuration, prompt construction, trigger, repair setting, scheduler, and budget policy; the intended intervention is admission. E/D share admission and differ only in state selection. Controlled ChatAFL retains its scheduling lineage; replacing it with the LoopFuzz scheduler would change the baseline. B–A is not an isolated admission comparison.

Proposal control has two scopes. A paired candidate diagnostic evaluates the same recorded output from the same target/campaign snapshot under direct and gated admission with

Table 6: Required configurations for arms A–E.

Arm	LLM	Admission	Scheduler	Role
A: AFLNet	No	Native AFLNet	AFLNet	Non-LLM baseline.
B: Controlled ChatAFL	Yes	ChatAFL direct/open-loop	ChatAFL lineage	B–A: controlled LLM-lineage change.
C: LoopFuzz-direct	Yes	Parseable candidates directly durable	Fixed LoopFuzz	Admission counterfactual.
D: LoopFuzz-gated-fixed	Yes	Evidence gate + provisional queue	Same fixed LoopFuzz	D–C: admission effect.
E: LoopFuzz-gated-calibrated	Yes	Identical to D	Productivity calibration	E–D: calibration effect.

Table 7: Targets, protocols, and build metadata.

Target	Protocol	Commit/version	Image digest	Coverage build / reset hook
LightFTP	FTP			
bftpd	FTP			
ProFTPD	FTP			
Pure-FTPd	FTP			
Exim	SMTP			
Live555	RTSP			
Kamailio	SIP			
Forked-daapd	DAAP/DACP			
Lighttpd1	HTTP			

identical descendant budgets. It supports a candidate-level counterfactual. Independent online campaigns diverge in queue and state history, so matching policies cannot guarantee identical prompt bytes or model outputs throughout. Recorded-action replay supports parser checks, predicate replay, snapshot comparisons, and provider-free artifact inspection; it does not make a replayed campaign equivalent to live adaptation.

6.3. Targets, builds, and resource controls

The primary design contains nine servers: four FTP servers, Exim (SMTP), Live555 (RTSP), Kamailio (SIP), Forked-daapd (DAAP/DACP over HTTP), and Lighttpd1 (HTTP). Exact build metadata unavailable for a verified comparison remains blank in Table 7. Hardware settings and version pins are not carried over from the old manuscript without evidence linking them to the present runs.

Each causal comparison uses the same source revision, flags, instrumentation, seeds, replay procedure, and reset policy. Launch order is randomized within host blocks and balanced across machines and periods. Runs record fuzzer seeds, quotas, measured CPU use, throttling, and execution rates. Container quotas alone do not establish equal CPU time. Startup, model waiting, admission trials, provisional descendants, and cleanup enter the resource ledger.

Each arm initially receives 10 independent 24-hour runs per target; C/D/E receive 10 more, reaching 20 each. For nine targets and one allocated CPU per run, the stages require 10,800 and 6,480 CPU-hours, totaling 17,280 before overhead/failures. These are planned resources, not measured usage. A four-target pilot covers Forked-daapd, Lighttpd1, LightFTP, and ProFTPD or Pure-FTPd with five arms, three runs, and four to six hours each. Pilot observations are excluded from formal inference.

The intention-to-run ledger includes planned, launched, completed, timeout, target/fuzzer crash, infrastructure failure, extraction failure, excluded, and rerun states, with reasons and linked attempts. Poor coverage is never an exclusion criterion. Only predefined infrastructure failures permit reruns, and original launches remain visible. An interrupted run is not silently extended to 24 hours. It contributes a declared failure outcome or censored observation according to the estimand, with sensitivity analysis; completed-run-only results are not intention-to-run results.

6.4. LLM fairness and configuration

The main design uses equal policy and equal ceilings. All LLM arms must use identical model configurations, intervention policies, and resource ceilings. Actual calls and token usage are reported because adaptive trajectories can trigger different numbers of interventions. Table 8 does not invent unfrozen settings.

An output-token cap per request is not a total campaign ceiling. A zero legacy token-cap field must be interpreted through its actual enforcement, not reported as equal-token control. Dollar costs require auditable usage and applicable prices; unknown charges remain blank.

Secondary comparisons report code-branch gain per 1,000 total tokens, per LLM call, at a common cumulative token budget, and at a common call count. Security costs use CPU-hours and 100,000-token units. Accounting includes unsuccessful generations, retries, validation, and optional repair. A fixed intervention schedule provides a cost-control sensitivity, not a replacement for the adaptive main policy. Ratios with zero denominators are undefined.

Table 8: Shared LLM settings required for B–E.

Field	Frozen value	Required handling
Provider / endpoint / immutable model version		Resolved immutable version; alias insufficient.
Temperature by call class		Match each call class across arms.
Top- p / presence and frequency penalties		Explicit values or documented defaults.
Maximum output tokens per attempt		Shared cap; log truncations/failures.
Maximum attempted calls / total tokens		Count all call classes and retries.
Timeout / retry count / backoff		Fixed policy; log waits/failures.
Model-side seed / support status		Record support; no determinism guarantee.
Prompt corpus / template version		Shared information policy; prompt/reply hashes.
Plateau cadence / threshold / reset		Shared trigger/reset, including state-growth dependencies.

6.5. Mechanism and calibration metrics

For every target/arm, report candidate generation, $P/U/R$ first-failure distribution, direct durable and provisional rates, conversion, expiration, promotion latency, queue size, trial latency, descendant executions, code gain per candidate/token, episode productivity, and throughput. Denominators and incomplete/censored events accompany rates.

Downstream productivity uses a fixed descendant horizon rather than admission gain. For a declared cohort C of N candidates enrolled before their outcomes are known, let $Y_i(H) = 1$ if a later descendant produces new code coverage within H executions, and 0 if the full horizon is observed without gain. With complete observation, define

$$\text{Productivity@ } H = \frac{1}{N} \sum_{i \in C} Y_i(H). \quad (13)$$

Initially $H = 64$. The admission trial is excluded. A positive outcome can be established early; a negative requires the complete horizon. With n_+ known positives and $n_?$ censored unresolved candidates, report the cohort bounds $[n_+/N, (n_+ + n_+)/N]$ rather than deleting incomplete negatives from the denominator. The primary paired diagnostic reserves the same full validation horizon for both controllers; incomplete online cohorts are reported with these bounds and censoring counts. Durable and provisional cohorts are distinct; post-promotion productivity uses a fresh horizon. Thus neither admission gain nor selective observation mechanically raises measured productivity.

Avoided queue pollution requires C–D evidence. In the paired diagnostic, identify direct-arm durable candidates without downstream gain and compare their retention under gating. D-arm rejections are not automatically pollution. A random, predeclared sample of rejected candidates receives shadow validation to estimate false negatives, with inclusion probabilities, snapshots, lineage, and cost recorded.

For complete episodes, $p_t = \hat{\theta}_{s,t}^{\text{pre}}$ predicts later reward r_t . The Brier score (BS) [18] is

$$\text{BS} = \frac{1}{n} \sum_{t=1}^n (p_t - r_t)^2. \quad (14)$$

Expected calibration error (ECE) [19] uses ten fixed equal-width probability bins and reports counts, mean predictions, and observed frequencies. Empty bins stay empty. Reliability plots

and the area under the precision–recall curve (AUPRC) [20] accompany the BS; with no positive episodes AUPRC is undefined. The archived diagnostics summarize AUPRC using non-interpolated average precision (AP). Compare the BS to a prequential base-rate predictor because rare rewards can make uninformative near-zero predictions look accurate. IPSM state-edge/code-gain ratios with zero code gain are undefined or shown as separate zero-gain cases.

Episodes within a run are dependent; uncertainty is clustered by run and reported by target/phase. Posterior trajectories and state weights are diagnostic, not evidence that every state is accurately learned. Correlation between IPSM growth and later code gain is evaluated within target and phase, without pooling raw counts across unlike programs.

6.6. Coverage and statistical analysis

Primary coverage outcomes are code-branch AUC over 24 hours and final code-branch count. AUC uses each run’s cumulative time series with a common interpolation convention and instrumentation, in branch-hours. Native endpoints at unequal durations are not substituted. Time to 95% of a target-specific common reference endpoint uses a threshold frozen before formal comparisons; unreached thresholds are censored.

Per target, report mean, median, dispersion, bootstrap 95% confidence intervals (CIs) [21], two-sided Mann–Whitney U tests [22], and Cliff’s δ [23]. The primary family comprises 36 coverage tests across nine targets: D–C and E–D contrasts for final branches and AUC. The Benjamini–Hochberg procedure [24] controls this family at $q = 0.05$. The primary admission-mechanism family has nine D–C productivity contrasts and is reported separately. Other diagnostics and sensitivity tests are exploratory. Effect direction is treatment minus comparator.

Cross-target summaries use equal-weight target effects and a hierarchical bootstrap resampling targets, then runs within targets. Raw branch counts from different programs or individual executions are not pooled as independent samples. Target regressions and uncertainty accompany aggregates. Historical time-to-trigger uses Kaplan–Meier estimates [25], right censoring, and CVE-specific deadline recall. Fuzzer-failure interruptions are distinguished because censoring may be informative.

Table 9: Available runs by target; D-arm status and duration.

Target	AFLNet	ChatAFL	LoopFuzz(D)	Completed(D)	Exit 137(D)	Duration (min)
LightFTP	13	11	15	15	0	1473–1522
bftpd	10	10	10	10	0	1499–1510
ProFTPD	10	10	12	12	0	1489–1518
Pure-FTPd	11	10	10	10	0	1460–1472
Exim	10	10	10	10	0	1477–1486
Live555	10	10	12	12	0	1489–1505
Kamailio	10	10	12	12	0	1472–1518
Forked-daapd	10	10	12	12	0	1474–1524
Lighttpd1	10	10	11	11	0	1469–1523

7. Available Evidence and Results

The empirical source for this revision is the supplied `Key_Experiment` directory. Earlier selected-run tables, significance counts, auxiliary comparisons, crash-class totals, and curves are not treated as measurements of the new controller. A blank numeric cell below means unavailable or unvalidated evidence, never a measured zero.

7.1. Archive inventory and observation horizon

The current extraction contains 94 AFLNet (A), 91 ChatAFL (B), 104 benchmark LoopFuzz (D), 90 direct-admission (C), and 86 calibrated (E) summary rows. The sensitivity groups contain 108 rows at $\gamma = 0.99$ and 55 rows at $\gamma = 1.0$. Extra rows are retained as archive observations without assuming independence; the nominal target is ten runs, while the observed count n remains explicit. Means and sample SDs use observed values only; unavailable runs are left unfilled.

All 104 benchmark LoopFuzz D summaries are labeled completed; status counts are shown per target in Table 9. The exit-code labels do not independently establish out-of-memory termination or a vulnerability. Runtime is fuzzer runtime, not total CPU time or necessarily the full startup-to-cleanup campaign duration.

Benchmark LoopFuzz supplies arm D and is counted once; it is the supplied equivalent of `ablation/gated_fixed`. Arms C and E use `ablation/direct` and `ablation/calibrated`, respectively. Two additional groups record sensitivity runs with $\gamma = 0.99$ and $\gamma = 1.0$. The startup logging-order issue in Table 4 means these declarations require launcher/end-record corroboration, and heterogeneous run counts prevent a complete matched-arm claim. No calibrated-arm effect is inferred from posterior fields alone.

The complete target–arm ledger, including both gamma sensitivity groups, is reported in Appendix A, Table A.1.

Table 10 retains every available A and D summary row. Its native terminal `b_abs` is taken from the supplied run summary; its IPSM state-edge count is likewise the supplied terminal summary value. All 628 selected archives have trajectory endpoints matching their supplied summary `b_abs` values; the trajectory and endpoint tables therefore use the same recorded terminal observation. These observations are descriptive, not matched 24-hour outcomes. Sample SDs express variation among the supplied rows; neither causal claims nor significance tests are attached to unequal-duration endpoints.

7.2. RQ1: Current alignment observations

Forked-daapd has mean native code-branch endpoints of 2330.8 ± 60.0 for AFLNet and 2670.2 ± 87.3 for recorded LoopFuzz, while IPSM state-edge endpoints are 21.6 ± 2.1 and 22.8 ± 2.9 . This direction does not reproduce the older motivating claim of code regression. The difference cannot be attributed to calibration, which is not established as active.

Lighttpd1 increases in both native metrics: code branches from 1940.7 ± 21.2 to 2126.7 ± 15.7 , and IPSM state edges from 17.7 ± 2.5 to 28.9 ± 2.7 . These aggregate observations do not demonstrate the hypothesized small state expansion with hidden code progress. LightFTP has near-equal code endpoints, 71.0 ± 0.0 versus 71.7 ± 2.3 , with larger recorded LoopFuzz IPSM state-edge counts, 187.3 ± 11.3 versus 207.0 ± 17.0 . That pattern motivates a saturation analysis but does not establish the full time course or identify individual misleading states.

All three cases require common-build, common-horizon verification and event joins before becoming mechanism evidence. The available episode fields do support a narrower diagnostic: Fig. 4 compares the logged pre-update probabilities with the logged binary coverage-save reward, separately for each target. All 31,826 E-archive records pass finite-probability, Beta-mean, binary-reward, episode-ID uniqueness, chronological-order, and within-episode discounted-update checks. Excluding 6,070 zero-mutation records leaves 25,756 positive-mutation records from 86 runs. Three between-episode posterior discontinuities are retained and documented; no missing history is reconstructed. The archived arm mapping does not by itself verify that every run used the intended scheduling policy.

Table 12 reports the BS, ten-bin ECE, and AUPRC as AP; lower BS/ECE is better. Its prequential reference uses only past included same-run rewards with Beta(1,1) smoothing. Percentile 95% CIs use 2,000 bootstrap resamples of whole runs within each target; they do not resolve unknown dependence between launches. Empty bins have no estimate, and bins represented by fewer than two runs have no CI. In Figure 4, points pair the bin-mean pre-update prediction with reward frequency; the diagonal denotes perfect calibration. Lower strips give positive-mutation episode counts, with actual runs n , episodes m , and nominal $N = 10$. The saved reward counts native coverage-save events and can reflect hit-count novelty, and positive mutation counts do not prove a complete fixed-energy episode. Consequently, these diagnostics do not fill the independent source-branch calibration outcomes in Table 14.

Table 10: Native A/D code and IPSM endpoints (mean $\pm$ SD).

Target	Code branches (b_abs)		IPSM state edges	
	AFLNet	LoopFuzz(D)	AFLNet	LoopFuzz(D)
LightFTP	71.0 $\pm$ 0.0	71.7 $\pm$ 2.3	187.3 $\pm$ 11.3	207.0 $\pm$ 17.0
bftpd	486.0 $\pm$ 8.8	502.7 $\pm$ 7.5	163.1 $\pm$ 15.4	226.7 $\pm$ 14.0
ProFTPD	5053.8 $\pm$ 97.4	5428.1 $\pm$ 104.0	190.5 $\pm$ 27.9	305.3 $\pm$ 20.3
Pure-FTPD	1076.4 $\pm$ 89.6	1223.3 $\pm$ 29.7	236.5 $\pm$ 26.7	321.4 $\pm$ 11.1
Exim	3798.8 $\pm$ 43.3	3914.9 $\pm$ 20.9	88.8 $\pm$ 5.2	152.7 $\pm$ 9.3
Live555	2893.6 $\pm$ 14.5	3067.1 $\pm$ 29.5	96.1 $\pm$ 8.6	170.1 $\pm$ 11.1
Kamailio	10002.6 $\pm$ 144.2	10631.2 $\pm$ 472.9	109.4 $\pm$ 9.7	128.2 $\pm$ 26.6
Forked-daapd	2330.8 $\pm$ 60.0	2670.2 $\pm$ 87.3	21.6 $\pm$ 2.1	22.8 $\pm$ 2.9
Lighttpd1	1940.7 $\pm$ 21.2	2126.7 $\pm$ 15.7	17.7 $\pm$ 2.5	28.9 $\pm$ 2.7

Table 11: Logged D-arm events; disposition labels do not verify queue membership.

Target	Candidates	Trials	Reject	Provisional	Durable	Episodes
LightFTP	48	48	44	0	4	13604
bftpd	13	13	13	0	0	3249
ProFTPD	0	0	0	0	0	2395
Pure-FTPD	173	173	166	0	7	5378
Exim	24	24	24	0	0	2650
Live555	851	851	844	0	7	7687
Kamailio	253	253	249	0	4	9004
Forked-daapd	412	411	410	0	1	12056
Lighttpd1	453	453	453	0	0	21570

Table 12: E-arm logged-reward calibration (95% CIs in brackets).

Target	n	Episodes	BS	Reference BS	ECE (10 bins)	AUPRC (AP)
LightFTP	10	6,622	0.188	0.183	0.166	0.452
			[0.178, 0.199]	[0.173, 0.196]	[0.161, 0.172]	[0.432, 0.472]
bftpd	10	2,643	0.249	0.247	0.105	0.517
			[0.243, 0.257]	[0.243, 0.251]	[0.094, 0.118]	[0.493, 0.541]
ProFTPD	10	1,002	0.183	0.144	0.137	0.810
			[0.179, 0.188]	[0.126, 0.157]	[0.106, 0.173]	[0.782, 0.846]
Pure-FTPD	10	2,848	0.227	0.243	0.053	0.700
			[0.218, 0.234]	[0.233, 0.248]	[0.041, 0.071]	[0.674, 0.721]
Exim	10	1,057	0.230	0.219	0.126	0.703
			[0.222, 0.239]	[0.211, 0.227]	[0.107, 0.149]	[0.674, 0.733]
Live555	10	2,259	0.224	0.218	0.082	0.742
			[0.212, 0.233]	[0.206, 0.229]	[0.065, 0.104]	[0.713, 0.771]
Kamailio	10	6,374	0.163	0.161	0.042	0.858
			[0.153, 0.171]	[0.149, 0.173]	[0.032, 0.056]	[0.847, 0.866]
Forked-daapd	6	754	0.246	0.252	0.093	0.622
			[0.236, 0.258]	[0.249, 0.256]	[0.063, 0.133]	[0.587, 0.643]
Lighttpd1	10	2,197	0.231	0.251	0.085	0.665
			[0.225, 0.237]	[0.249, 0.253]	[0.072, 0.097]	[0.638, 0.687]

7.3. RQ2: Observed admission records and missing counterfactual

For the benchmark LoopFuzz D archive, the event stream contains 2,227 candidate records, 2,226 trial records, 2,203 reject labels, 23 durable labels, and no provisional labels. One candidate lacks a trial match; no trial lacks a candidate match. The corresponding C and E totals are 1,326/1,326 and 735/734 candidates/trials, respectively, with 1,326 direct durable labels for C and 725 rejects plus 9 durable labels for E. These are logged decisions rather than verified queue membership, and they do not demonstrate the required 99.9% event completeness.

Candidate/admission files occur in the C, D, and E archives with target-specific gaps; ProFTPD has no candidate/admission records in both D and E despite nonzero model-call counters.

The supplied archives lack provisional and repair lifecycle files. Table 11 reports the D benchmark counts, while Table A.1 gives the arm endpoints. Zero recorded lifecycle events do not yield a zero conversion rate or prove absence of repair/provisional behavior.

Disposition labels are not independently verified queue membership. The native-save ordering and state-only reachability problems prevent interpreting rejected labels as avoided queue pollution or durable labels as proof of strict predicate conformance. Without a validated C–D comparison and descendant lineage, downstream productivity, shadow-validation false negatives, and avoided nonproductive retention remain blank. Logged trial flags and latency can nevertheless be summarized descriptively in Table 14; Fig. 7 visualizes the reconciled record

E archives: pre-update probability versus logged reward

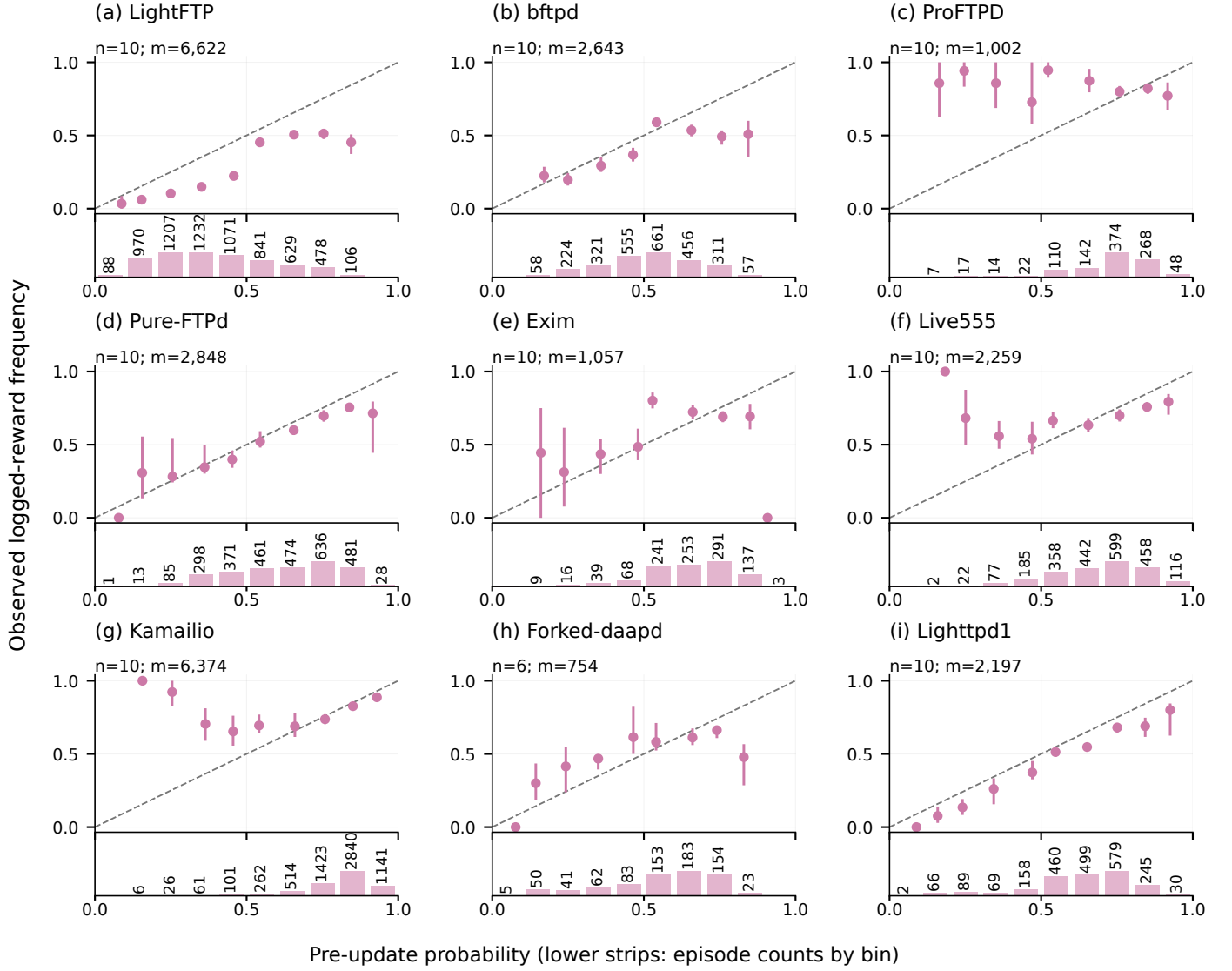


Figure 4: E-arm logged-reward calibration. Points: bin means; bars: 95% CIs.

counts. A first-failure flag is a logged predicate result, not an independently validated reason for retention or rejection.

7.4. RQ3: Scheduling evidence and unfilled causal results

The supplied C, D, and E archives contain 56,164, 77,593, and 31,826 state-episode records, respectively. Their logged episode mutations counts range from zero to 18,816, which is not a shared fixed-energy measurement surface. Some coverage fields are named as code branches or code edges while the inspected implementation fills bitmap-slot counts or coverage-save-event counts. These values cannot be merged with source-level `b_abs` or treated as the main reward of Equation 4.

The archive contains E-arm calibrated runs and two gamma sensitivity groups, but the logged implementation and heterogeneous horizons do not establish that all runs used the intended Thompson policy or that D–E is a matched causal comparison. Table 13 reports the recoverable endpoint/AUC values;

all 45 main target–arm combinations are represented, including ten E/bftpd archives. The two gamma sensitivity groups lack Forked-daapd archives. Calibration benefits still require a frozen, matched-horizon analysis.

Fig. 5 shows code-branch medians and interquartile ranges on a five-minute grid over 0–24 hours from fuzzer startup; the bands are not CIs. Linear interpolation stays within recorded observations, and labels give the contributing n at 24 hours. Table 13 instead integrates each full observed trajectory in branch-hours; cells show branches with actual n above AUC, both as mean $\pm$ SD. Each run contributes only between its first and last observation; time-dependent denominators accompany the figure data. Changes in the set of contributing runs can change an aggregate curve even when individual histories do not decrease. In 127 of the 465 source trajectories, at least one recorded branch count decreases. The figure preserves these observations with-

Table 13: Branch endpoints (top) and AUCs (bottom), mean $\pm$ SD.

Target	A	B	C	D	E
LightFTP	71.0 $\pm$ 0.0 (n=13)	71.3 $\pm$ 0.9 (n=11)	80.6 $\pm$ 15.5 (n=10)	71.7 $\pm$ 2.3 (n=15)	73.9 $\pm$ 4.5 (n=10)
	1758.5 $\pm$ 33.8	1728.1 $\pm$ 36.5	1987.7 $\pm$ 394.1	1780.7 $\pm$ 45.2	1825.0 $\pm$ 107.6
bftpd	486.0 $\pm$ 8.8 (n=10)	492.2 $\pm$ 14.1 (n=10)	498.9 $\pm$ 12.2 (n=10)	502.7 $\pm$ 7.5 (n=10)	503.7 $\pm$ 7.1 (n=10)
	11868.7 $\pm$ 165.6	12068.6 $\pm$ 323.8	12082.0 $\pm$ 451.0	12267.7 $\pm$ 387.9	12416.6 $\pm$ 220.9
ProFTPD	5053.8 $\pm$ 97.4 (n=10)	5205.9 $\pm$ 166.7 (n=10)	5446.3 $\pm$ 194.7 (n=10)	5428.1 $\pm$ 104.0 (n=12)	5469.8 $\pm$ 215.7 (n=10)
	122652.5 $\pm$ 1730.1	125217.7 $\pm$ 2989.1	130538.5 $\pm$ 4511.6	132539.2 $\pm$ 2868.9	130564.7 $\pm$ 4604.1
Pure-FTPD	1076.4 $\pm$ 89.6 (n=11)	1109.3 $\pm$ 40.0 (n=10)	1221.2 $\pm$ 48.9 (n=10)	1223.3 $\pm$ 29.7 (n=10)	1215.4 $\pm$ 63.7 (n=10)
	23054.1 $\pm$ 2084.4	24976.6 $\pm$ 1126.8	28860.8 $\pm$ 1645.7	27863.9 $\pm$ 1496.9	27894.3 $\pm$ 1737.6
Exim	3798.8 $\pm$ 43.3 (n=10)	3761.7 $\pm$ 60.4 (n=10)	3896.1 $\pm$ 25.6 (n=10)	3914.9 $\pm$ 20.9 (n=10)	3858.3 $\pm$ 32.8 (n=10)
	91083.0 $\pm$ 1031.5	91301.7 $\pm$ 1501.4	94880.1 $\pm$ 928.3	93659.0 $\pm$ 573.1	93123.6 $\pm$ 924.6
Live555	2893.6 $\pm$ 14.5 (n=10)	2882.4 $\pm$ 22.0 (n=10)	3075.7 $\pm$ 34.4 (n=10)	3067.1 $\pm$ 29.5 (n=12)	3054.2 $\pm$ 20.5 (n=10)
	71127.4 $\pm$ 517.3	71224.8 $\pm$ 422.5	73511.0 $\pm$ 562.9	75436.1 $\pm$ 712.2	72378.5 $\pm$ 1049.8
Kamailio	10002.6 $\pm$ 144.2 (n=10)	10644.3 $\pm$ 96.5 (n=10)	10800.6 $\pm$ 129.0 (n=10)	10631.2 $\pm$ 472.9 (n=12)	10830.0 $\pm$ 91.3 (n=10)
	242853.7 $\pm$ 2772.6	257606.8 $\pm$ 2082.0	260260.9 $\pm$ 2942.4	254332.1 $\pm$ 9492.1	259377.2 $\pm$ 2649.6
Forked-daapd	2330.8 $\pm$ 60.0 (n=10)	2169.6 $\pm$ 113.0 (n=10)	2585.4 $\pm$ 86.2 (n=10)	2670.2 $\pm$ 87.3 (n=12)	2648.2 $\pm$ 116.0 (n=6)
	54453.2 $\pm$ 1494.1	49393.2 $\pm$ 3335.9	61776.8 $\pm$ 1863.7	63646.7 $\pm$ 1977.0	62349.5 $\pm$ 1726.9
Lighttpd1	1940.7 $\pm$ 21.2 (n=10)	1884.1 $\pm$ 25.3 (n=10)	2145.0 $\pm$ 69.8 (n=10)	2126.7 $\pm$ 15.7 (n=11)	2068.1 $\pm$ 44.5 (n=10)
	46909.4 $\pm$ 235.7	45075.8 $\pm$ 729.3	53003.8 $\pm$ 1613.7	51223.2 $\pm$ 621.1	50385.1 $\pm$ 941.3

Table 14: Logged C/D/E mechanisms; blank cells indicate unvalidated evidence.

Metric	C	D	E	Denominator / interpretation
Candidate records	1,326	2,227	735	Unique candidate IDs per archive.
Trial records	1,326	2,226	734	Unique trial IDs per archive.
Reject / durable labels	0 / 1,326	2,203 / 23	725 / 9	Logged trial dispositions.
First failed logged <i>P/U/R</i>	0 / 1,208 / 0	0 / 2,071 / 0	0 / 687 / 0	First false <i>P</i> , then <i>U</i> , then <i>R</i> ; denominator: trials.
Direct durable rate	100.0% (1,326/1,326)	1.0% (23/2,227)	1.2% (9/735)	Durable labels / generated candidates.
State-only provisional rate				All candidates; disabled for C.
Provisional conversion / expiration				Provisional admissions; include pending/censored cases.
Trial latency (ms)	439.2 $\pm$ 405.9 (<i>n</i> = 67)	403.6 $\pm$ 574.6 (<i>n</i> = 81)	787.3 $\pm$ 552.9 (<i>n</i> = 60)	Within-run mean <code>latency_ms</code> : mean $\pm$ SD across runs with trials.
Promotion latency				Verified queue-promotion timestamps required.
Downstream productivity@64				Descendant code gain; exclude admission gain.
Queue size / nonproductive retention				Queue state and C–D counterfactual.
Shadow-validation false-negative rate				Random rejected-candidate sample.
Code-branch gain per episode				Completed equal-energy episodes.
BS / ECE / AUPRC				Fixed-energy code-branch reward; logged diagnostics: Table 12.
Code gain per candidate / 1,000 tokens				Joined code gain and complete usage.

out applying a cumulative maximum or otherwise correcting the replay output; both source decreases and changing support can produce aggregate decreases.

Fig. 6 adds three reproducible state-level archive illustrations. For each motivating target, the selected E run is the lower-median run by terminal branch count, with source path breaking ties; its two states with the most positive-execution episodes are shown. This rule uses no calibration score or effect direction. The complete histories, including zero-execution episodes, show pre-update probabilities above the final-selection/frontier-score ratio. Time denotes episode end; lines guide the eye and colors identify states within each target. Zero-frontier ratios are undefined and omitted. These cases illustrate recorded behavior and do not identify a causal response-state/branch mismatch.

7.5. RQ5: Recorded costs and interpretation limits

The final saved statistics across the benchmark LoopFuzz D archives report model calls and token counts by target in Table 15; the C/E arm ranges and means are retained in the machine-readable arm summary. Saved model-call counts range from 52 to 248 per D run; the logged cap of 64 therefore does not describe an enforced all-call campaign ceiling. These are saved tool counters, not independently reconciled provider billing. Some final aggregate statistics lag the event stream, so costs may omit the last reporting window.

Recorded configurations include a model alias, grammar/plateau temperatures of 0.5/1.2, top-*p* 1.0, maximum output tokens 4096, and total-token-cap field 0. The alias does not establish an immutable provider model. Startup configuration

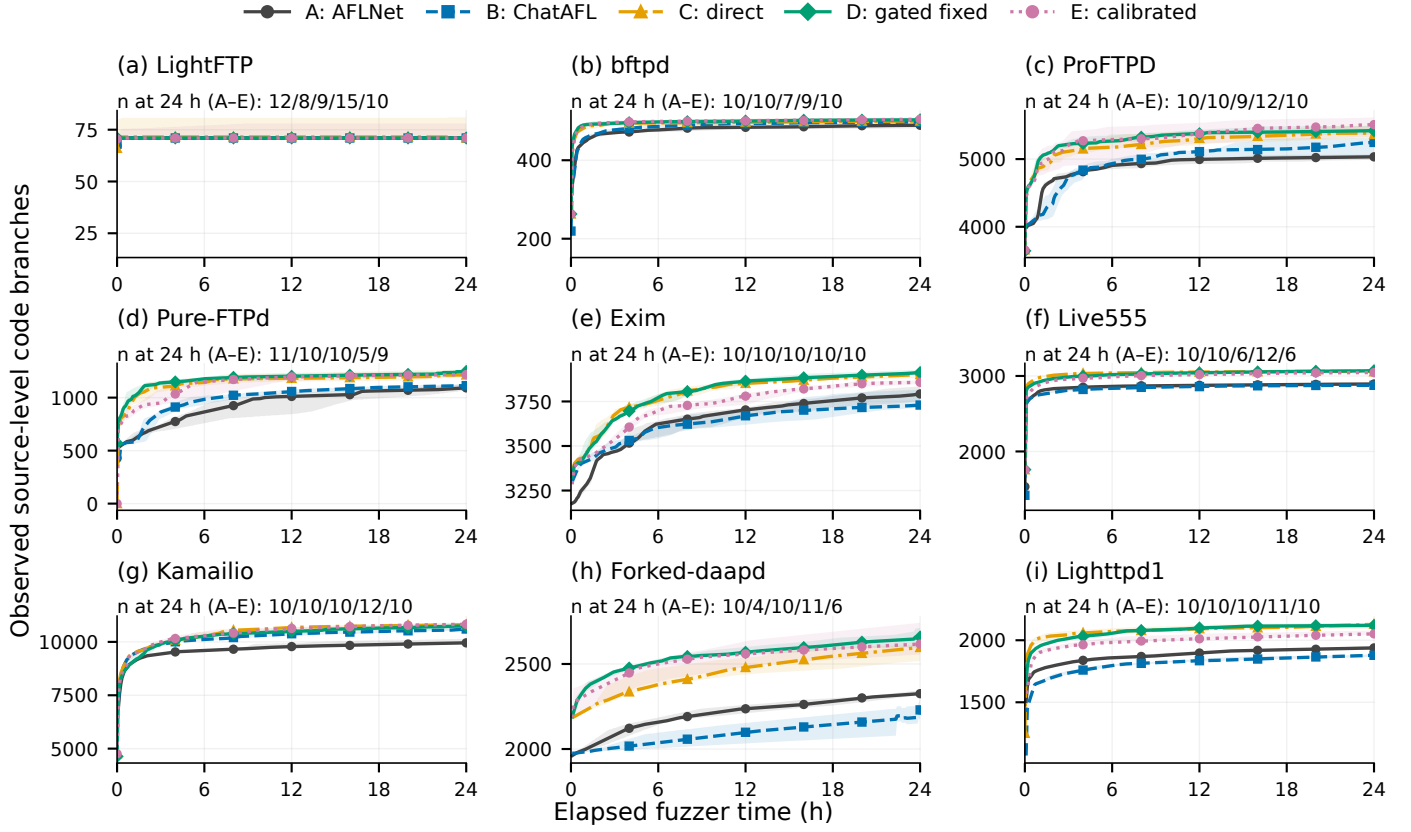


Figure 5: A-E code-branch trajectories: medians and interquartile ranges.

Recorded posterior and scheduling multiplier: three E-archive cases
Selection: lower-terminal coverage run, then two most observed states

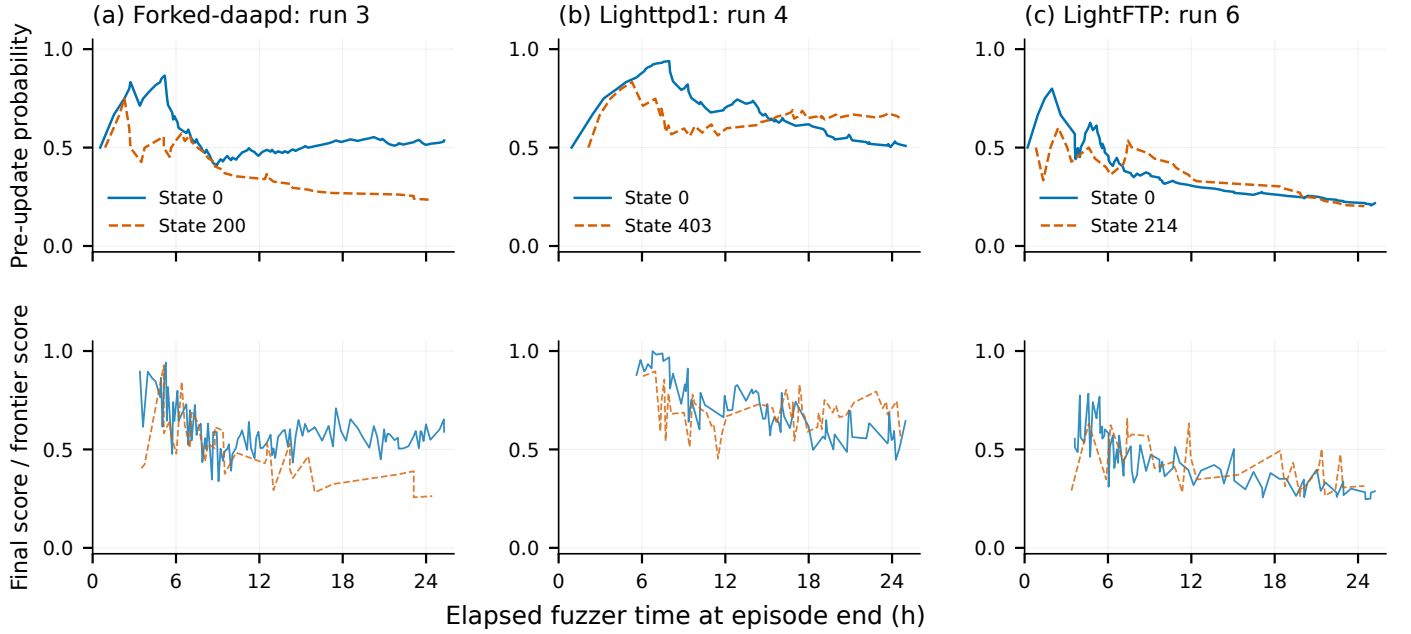


Figure 6: E-arm state histories: pre-update probabilities (top) and scheduling multipliers (bottom).

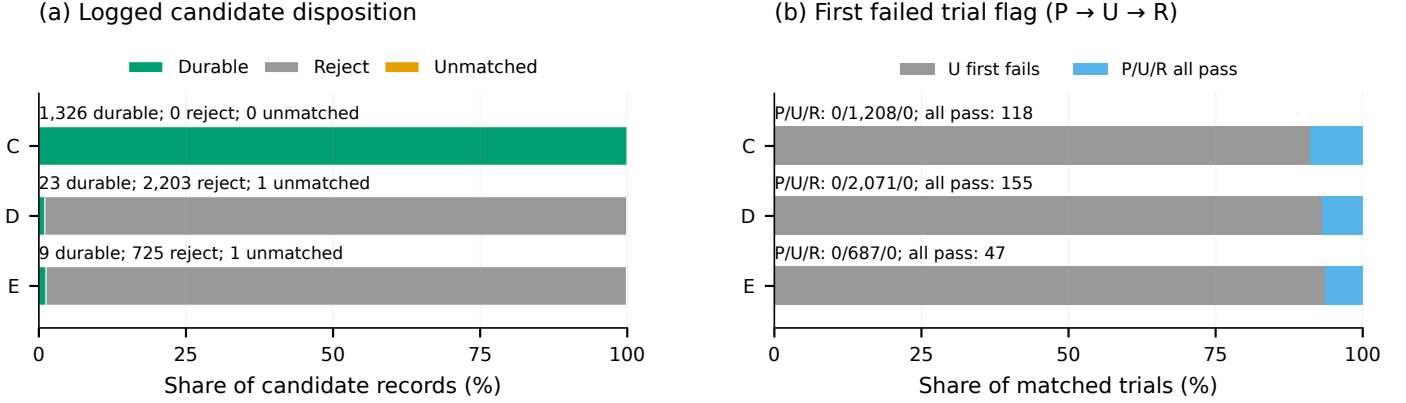


Figure 7: C/D/E logged dispositions (left) and first failed $P/U/R$ predicates (right).

Table 15: Saved D-arm costs: per-run ranges by target.

Target	Model calls	Total tokens (k)
LightFTP	59–161	106.0–224.7
bftpd	199–248	198.0–301.8
ProFTPD	188–192	170.1–198.8
Pure-FTPd	207–242	200.2–264.1
Exim	106–123	109.4–151.0
Live555	123–189	264.1–433.1
Kamailio	56–108	157.0–285.5
Forked-daapd	52–163	103.4–540.9
Lighttpd1	131–187	168.5–252.5

Table 16: Optional repair diagnostics; unavailable values remain blank.

Metric	E, repair off	E, repair on
Repair opportunities / attempts		
Before/after validation success		
Post-repair descendant code gain		
Changed fields / locality		
Token / latency cost		
Independent effect / interval		

timing and missing build/resource provenance limit interpretation. These historical values do not fill the prospective shared configuration in Table 8.

Candidate token fields do not necessarily cover all call classes or retries; summing them is not total campaign cost. Low-activation targets, high-rejection targets, token outliers, saturation, and regressions must remain in the final analysis. Low activation may reflect productive mutation or a restrictive trigger; high rejection may reflect either useful gating or false negatives. Both need mechanism evidence.

Figures 8 and 9 plot saved tokens and model calls against terminal code branches for 371 B–E runs, using target-specific scales. Tokens combine prompt and completion counts and are displayed in thousands (k). AFLNet has no comparable saved counter; calls measure neither token expenditure nor CPU time. These views reveal dispersion and usage ranges without estimating a causal return per token or call. Measured CPU-hours and controlled A–E CVE recall remain unavailable. The case-level replays in Section 8 do not supply those cost–performance axes.

8. Vulnerability Replay Evidence and Rediscovery Protocol

The supplied `Key_Experiment/vulnerability` collection contains four case dossiers: three labeled with CVE identifiers and one SMARTPL availability case without a CVE. Each includes a report and a trigger input; three include vulnerable-side log excerpts, and ProFTPD also includes a patched-side replay log. Tables 17 and 18 report this evidence.

Identifiers, versions, and replay counts are attributed to these local records; the archive does not include the upstream patches or complete build manifests needed to independently certify every vulnerability/patch pair.

8.1. Case identity and evidence provenance

We distinguish an attached log observation (L) from a replay result stated only in a case report (R). A report can contain additional trials not represented by the supplied log excerpt. Repeated lines within a log are not independent trials. Input hashes and byte lengths are recorded in the companion `vulnerability_evidence_20260929.json`; no additional target executions were performed for this manuscript update. Table 17 identifies the recorded builds, triggering conditions, and differential evidence.

For ProFTPD, the report identifies an out-of-bounds read in `make_ftp_cmd`; the attached ASan trace records the corresponding heap-buffer-overflow signature. The oracle uses a 65,535-byte command buffer. The report states that the default 512-byte configuration does not expose the same error to ASan, so the outcome is conditional on this configuration. The patched log lists three rounds returning an FTP 500 response with no ASan report and normal one-shot exit. This supports a recorded differential replay; immutable image digests, build flags, and the actual applied patch remain unavailable in this collection.

For LIVE555, the supplied ASan excerpt contains the public-PoC use-after-free signature at offset `0x614b86`. Its two-line observation is repeated verbatim and is counted once as an attached signature. The report separately attributes fuzzer-generated inputs to the same allocation/free lifecycle, reports 8/9 replay hits, and records failure of a local patch attempt. The described

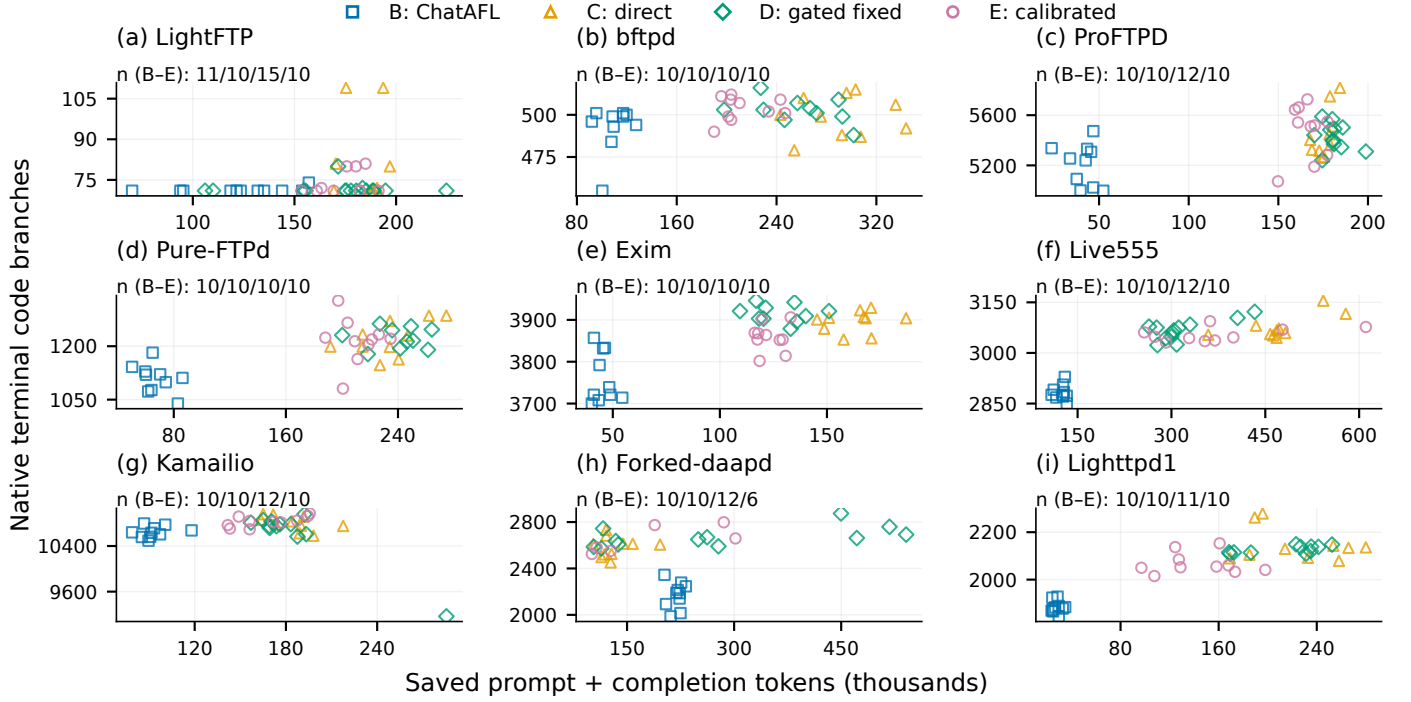


Figure 8: Saved tokens versus terminal code branches, by target and arm.

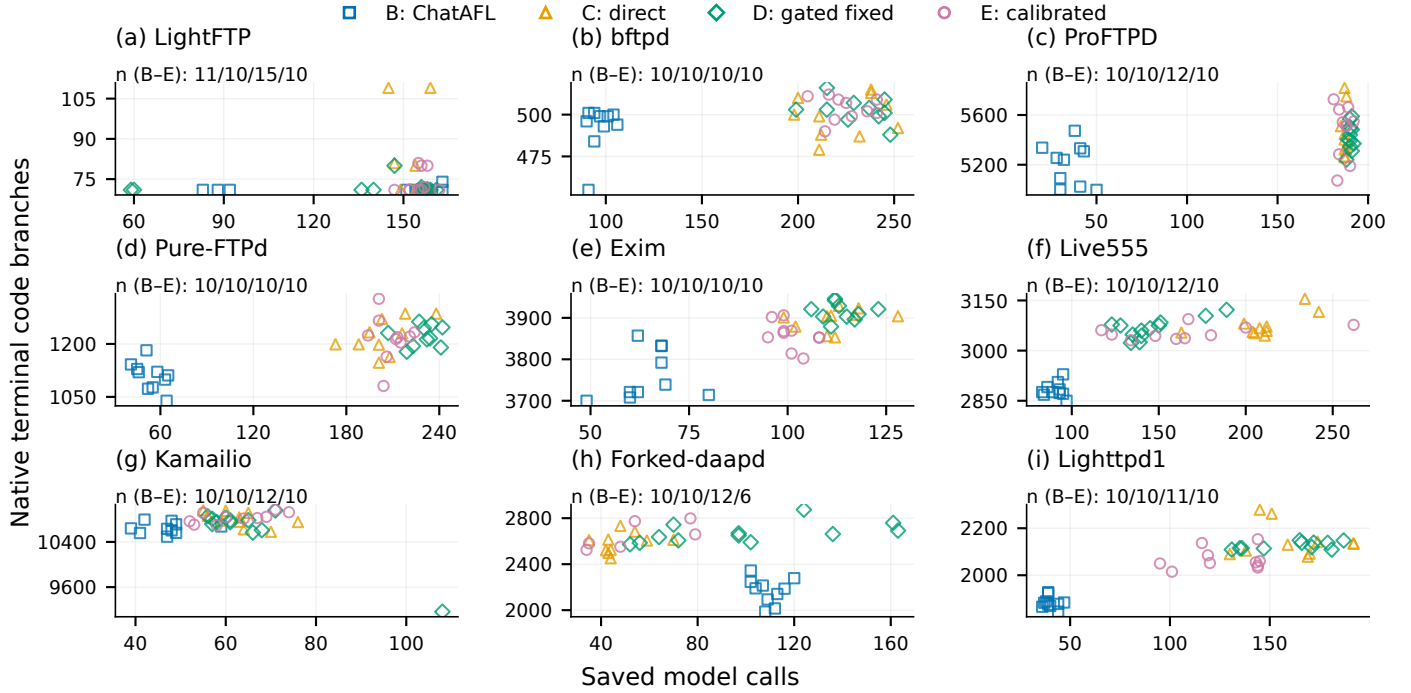


Figure 9: Saved model calls versus terminal code branches, by target and arm.

`trigger_loopfuzz.raw`, full allocation/free traces, and per-trial replay metadata are absent. The archived public PoC therefore supports replay diagnostics, but cannot demonstrate independent rediscovery by the evaluated controller or establish the cross-version CVE attribution on its own.

For Kamailio, the attached log records a negative parsed Content-Length, providing evidence for the reported overflow

predicate. This path requires TCP and cannot be evaluated through the UDP-only benchmark configuration. The latest report records suppression by a local guard, but supplies neither a patched-side log nor a repetition denominator. That observation is retained as R; it does not verify equivalence to the identified upstream patch. The report's earlier registry-only 20/20 versus 0/20 statement is excluded from the quantitative table because

Table 17: Vulnerability cases and archived evidence (L: log; R: report).

Case / interface	Reported vulnerable build	Trigger evidence / oracle	Patch or control evidence
CVE-2023-51713 ProFTPD / FTP	61e621e743346; ASan; 65,535-byte command buffer	60,006-byte input; heap-buffer-overflow: one-byte read beyond a 65,568-byte region (L).	Base + 97bbe68363cc; three rejections, no ASan (L); same input/buffer.
CVE-2026-38998 LIVE555 / RTSP	Campaign: 2023.05.10; public report: 2026.02.26 (R)	728-byte public PoC; heap-use-after-free (L). Fuzzer-discovered input absent.	Local handler-removal patch: 1/1 still triggers (R); upstream-patch differential absent.
CVE-2026-39863 Kamailio / SIP over TCP	a2209018fb; TCP listener required	278-byte input; parsed Content-Length -10 (L); overflow predicate, not crash count.	Local INT_MAX/10 guard suppresses fingerprint (R); reported fix 060b6c2530c8. No patched log/repeat count.
SMARTPL availability case forked-daapd / HTTP	Reported 27.2, 2ca10d9b	148-byte input; lexer errors; health probe fails; process alive (R). Replay log absent.	No patched replay/CVE; parser reportedly removed in 28.4.

Table 18: Reported replay batches and separate controls; original denominators retained.

Case	Vulnerable-side batches (R)	Patched or diagnostic control	Interpretation / missing evidence
CVE-2023-51713	1/1 seed; 1/1 fuzzer crash (Sept. 22); 3/3 batch crashes (Sept. 25).	Reported fix: 0/3 ASan activations; 3/3 FTP 500 rejections (L, Sept. 28).	Separate replay batches; no arm-specific discovery probability or trigger time.
CVE-2026-38998	1/1 public PoC; three fuzzer inputs: 3/3 each; later aggregate: 8/9.	Local patch: 1/1 still triggers (R); successful patched-side denominator absent.	Unpooled batches; fuzzer input/trial records absent; no confirmed patch pair.
CVE-2026-39863	1/1 seed; queue samples: 3/4 (60 s), 19/20 (60 min), 19/20 (1,540 min), 50/51 (separate 60-min batch).	Local guard: fingerprint absent (R); repeat count unavailable.	Queue samples, not campaign recall; no extrapolation to 195 entries; patched log absent.
SMARTPL availability case	5/5 advisory repeats; 3/3 48-message-seed repeats; 1/1 later advisory replay.	Sampled campaign hangs: 0/5 SMARTPL hits (R); patched replay absent.	Known-input unavailability; no confirmed campaign rediscovery or CVE.

the registry and trial records are not supplied. The SMARTPL report describes reproducible unavailability from a known input, while none of five sampled campaign hang inputs satisfies that predicate. This case has neither a CVE identity nor a verified patched counterpart.

8.2. Replay outcomes and their denominators

Table 18 preserves each reported batch separately. A numerator counts replay attempts or selected queue inputs satisfying that case’s predicate; the denominator is the number tested. These are not independent 12-hour fuzzer runs, and the main experiment’s nominal $N = 10$ is not substituted for the actual replay denominator. Selected batches, per-input repeats, and later aggregate counts may overlap; they are not pooled into one success rate. In particular, 19/20 selected inputs is not 95% campaign recall, and 8/9 LIVE555 replays is below the prospective 95% stability threshold.

The reports also list crash-file totals and campaigns outside the supplied benchmark/ablation ledger. These totals are not unique vulnerabilities, and durations such as 60 or 1,540 minutes do not identify time to first trigger. The collection lacks a join from each reproducing input to a verified A–E run, its model information history, and its independent oracle/root cause. We therefore do not assign these replay outcomes to individual arms,

claim newly discovered CVEs, or infer admission/calibration gains from them.

8.3. Controlled rediscovery still to be measured

The confirmatory benchmark requires a vulnerable base and the same base plus a minimal upstream security patch, with dependencies, instrumentation, configuration, and input compatibility held constant. Each case must have a network-reachable trigger, an independent reached/triggered oracle, reproducible builds/reset, and patch validation. The planned six-case pilot across at least three protocols, followed by expansion toward twelve, remains unfinished. Pilot qualification requires at least 95% observed oracle repeatability over a declared validation set, no patched-target activation, automated build/reset/cleanup, and information separation. The four dossiers above are case evidence, not six qualified benchmark pairs; a small all-success replay batch alone does not establish an underlying success probability of at least 95%.

Information-controlled campaigns must exclude CVE identifiers, advisories, public PoCs, and patch contents from model inputs and campaign seeds, while retaining anonymous oracle IDs in separate benchmark metadata. The public PoC and known triggers in this collection are validation inputs; their replay is not evidence of information-controlled rediscovery. Source-level

trigger conditions, sanitizer reports with patch validation, and protocol invariants distinguish reaching relevant code from actually triggering a defect. Experiments require isolated target networks and reset between trials; these are protocol requirements, not properties inferred from the reports.

For each CVE and arm, the formal plan retains ten independent runs and a 12-hour deadline, frozen before execution. Deadline recall, time-to-trigger, reached-but-not-triggered counts, patched-target activation, executions/coverage/tokens before trigger, and recall per CPU-hour or 100,000 tokens remain unavailable for the controlled A–E comparison. Nontriggers require right-censoring records, and interrupted runs require separate failure accounting. The supplied replay batches do not supply those risk sets, so Kaplan–Meier estimates or arm-level security-effectiveness claims are not reported.

9. Discussion

9.1. What calibration can and cannot repair

The controller calibrates the instrumental utility of an observed state proxy; it does not recover or verify semantic protocol state. If one response identifier aliases several server states, a single productivity estimate averages them. Reweighting the identifier cannot reconstruct lost distinctions. Richer parsing or internal-state instrumentation addresses different aspects at the cost of adapter complexity and deployment effort.

Low productivity can arise from a poor response proxy, saturated coverage, restrictive grammar, inadequate seeds, or insufficient energy. Independent reward prevents IPSM novelty from certifying itself but does not automatically diagnose these causes. The boundary analysis includes aliasing, cold start, phase, and target-specific regressions.

A fixed provisional horizon can discard a bridge whose payoff takes more descendants. Shadow validation and horizon sensitivity estimate this limitation; they do not prove every expired candidate worthless. Similarly, *U/R* can exclude useful error-handling inputs. Independent oracle retention and false-negative analysis are therefore necessary.

9.2. Cost, nonstationarity, and provider variability

Selection-indexed discounting can retain stale evidence for rarely selected states. Priors and the exploration floor preserve opportunities without guaranteeing fast identification. When code coverage saturates, nearly every reward may be zero, and calibration may offer little benefit.

Provider nondeterminism persists despite recorded prompts and fixed parameters. Aliases may resolve to different backends, retries alter costs, and target responses alter subsequent prompts. Equal-policy/equal-cap controls declared resources; actual usage and cost-matched analysis expose remaining tradeoffs. Provider-free replay audits the local controller rather than reproducing an adaptive campaign.

9.3. Decision criteria for unfinished experiments

Calibration remains a principal empirical claim only if the pilot produces nondegenerate predictions, reduces investment in branch-unproductive Forked-daapd states when observed, does not suppress Lighttpd1 solely for limited IPSM novelty, avoids severe throughput regressions on at least three pilot targets, and improves predictive diagnostics relative to a shared prequential base-rate predictor. D’s logged shadow productivity estimates provide an additional diagnostic; its raw queue-yield score is not treated as a calibrated probability. E–D differences require a plausible measured mechanism. These are development decisions, not confirmatory significance tests.

If these checks fail, calibration becomes exploratory and the empirical focus shifts to admission. If rediscovery does not improve, the result is reported without selecting easier CVEs after inspecting formal outcomes. Unsupported repair remains diagnostic. After raw-data and parameter freeze, revisions are limited to factual corrections, analysis/figure corrections, scoped confirmatory reruns, and artifact documentation; they do not introduce new algorithms or tune formal results into success.

The planned full evidence package requires clear D–C and E–D estimates, complete candidate-level mechanisms, at least nine targets, twenty runs per core arm, and a stable historical benchmark of approximately eight to twelve vulnerabilities. The four replay dossiers do not yet constitute that benchmark. These are completion criteria, not achieved sample counts.

10. Threats to Validity

Table 19 summarizes the controls and remaining evidence limits.

The audit is retrospective. The inspected source may differ from archived executables; a recorded short commit string alone does not verify binary identity. Source-level concerns and archive-level facts retain separate provenance. An absence of corrected significance does not establish equivalence, while favorable descriptive endpoints do not establish a mechanism under unresolved duration, resource, or implementation differences. Admission, calibration, and rediscovery each require their own evidence.

11. Conclusion

LoopFuzz treats response-derived protocol states and LLM output as uncertain evidence for resource allocation. Its controller contract separates code progress from IPSM novelty, gives state-only proposals bounded provisional validation, and calibrates state productivity at scheduling episodes. The response-visible interface remains useful, while independent code feedback determines its measured value.

The supplied benchmark and ablation archives support descriptive A–E endpoints, AUCs, time-resolved branch curves, saved-cost comparisons, and logged reward/admission diagnostics, together with a partial implementation audit. Four additional vulnerability dossiers provide case-level replay evidence, with attached logs distinguished from report-only trial counts

Table 19: Validity threats, required controls, and evidence limits.

Dimension	Required control	Current limitation
Internal	Admission-only C–D; scheduling-only E–D; repair off; paired snapshots.	Missing observations; implementation-ordering differences.
Construct	Separate IPSM state edges, code branches, bitmap counts, and reached/triggered events.	Native novelty/summaries incompletely measure the contract.
Statistical	Repeated trials; test families; effect sizes; clustered/hierarchical intervals; censoring.	Partial, unequal-duration data; no confirmatory effects.
Selection	Intention-to-run ledger; predefined infrastructure reruns.	Unlaunched attempts absent; extra rows do not prove independent launches.
Reproducibility	Build/image pins; seeds; prompts/replies; event joins; resource traces.	Aliases and incomplete provenance prevent exact replication.
External	Multiple targets/protocols; negative cases; adapter assumptions.	Encrypted/opaque binary protocols untested.
Security	Minimal patch pairs; independent triggers; information separation.	Controlled A–E outcomes and patch/build provenance incomplete; pretraining leakage unresolved.

and incomplete differentials. These observations do not yet establish causal admission gains, calibrated-scheduling gains, or improved historical rediscovery. The five-arm protocol, event schema, and explicitly unavailable outcomes identify what remains to be measured. Effectiveness claims must follow those measurements, including unfavorable targets and resource costs.

CRedit authorship contribution statement

Author contribution roles remain to be confirmed by all authors before submission.

Declaration of competing interest

The competing-interest declaration remains to be confirmed by all authors.

Funding

Funding information remains to be confirmed by all authors.

Data availability

This revision uses the locally supplied experimental archive as extracted on September 29, 2026, including four vulnerability dossiers with input hashes and evidence provenance. Its companion materials include the 628-run ledger, source hashes, numerical figure data, vector figures, and reproducible analysis and verification scripts. Public release location, access conditions, and permanent identifiers remain to be confirmed. The intended package contains build manifests, launch ledgers, raw event schemas, sanitized prompts/replies, analysis scripts, and patch-pair oracle metadata with information separation. No public archive or exact provider replay is claimed.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

OpenAI Codex assisted manuscript restructuring, language revision, LaTeX editing, figure-generation and analysis scripts, and consistency checks. The authors must verify the technical content, references, empirical interpretations, and final disclosure before submission.

Appendix A. Complete archive observations

Table A.1 retains the full archive inventory behind the descriptive results. The table uses the same observed values and sample SDs as the main results; repeated counts are shown once per row. Missing target–arm archives are not imputed. Row IDs combine arm and target number. Arms A–E retain their main-text meanings; E_{99} and E_{100} denote sensitivity groups at $\gamma = 0.99$ and $\gamma = 1.0$.

1: LightFTP 4: Pure-FTPd 7: Kmailio
2: bftpd 5: Exim 8: Forked-daapd
3: ProFTPD 6: Live555 9: Lighttpd1

AUC: branch-hours over each recorded trajectory. The IPSM column reports state-edge counts. Archive status codes: C=completed, X=exit_137_near_limit, S=sigabrt, O=oom_killed; labels do not verify failure causes.

Table A.1: Archive endpoints and observed-horizon AUC (mean $\pm$ SD).

ID	n	Branches	AUC	IPSM Status
A1	13	71.0 $\pm$ 0.0	1758.5 $\pm$ 33.8	187.3 $\pm$ 11.3 C:13
A2	10	486.0 $\pm$ 8.8	11868.7 $\pm$ 165.6	163.1 $\pm$ 15.4 C:10
A3	10	5053.8 $\pm$ 97.4	122652.5 $\pm$ 1730.1	190.5 $\pm$ 27.9 C:10
A4	11	1076.4 $\pm$ 89.6	23054.1 $\pm$ 2084.4	236.5 $\pm$ 26.7 C:11
A5	10	3798.8 $\pm$ 43.3	91083.0 $\pm$ 1031.5	88.8 $\pm$ 5.2 C:10
A6	10	2893.6 $\pm$ 14.5	71127.4 $\pm$ 517.3	96.1 $\pm$ 8.6 C:10
A7	10	10002.6 $\pm$ 144.2	242853.7 $\pm$ 2772.6	109.4 $\pm$ 9.7 C:10
A8	10	2330.8 $\pm$ 60.0	54453.2 $\pm$ 1494.1	21.6 $\pm$ 2.1 C:10
A9	10	1940.7 $\pm$ 21.2	46909.4 $\pm$ 235.7	17.7 $\pm$ 2.5 C:10
B1	11	71.3 $\pm$ 0.9	1728.1 $\pm$ 36.5	195.1 $\pm$ 14.6 C:11
B2	10	492.2 $\pm$ 14.1	12068.6 $\pm$ 323.8	167.1 $\pm$ 12.5 C:10

Table A.1 (continued)

ID	<i>n</i>	Branches	AUC	IPSM Status
B3	10	5205.9±166.7	125217.7±2989.1	150.0±18.5 C:10
B4	10	1109.3±40.0	24976.6±1126.8	186.0±31.5 C:10
B5	10	3761.7±60.4	91301.7±1501.4	115.8±13.5 C:10
B6	10	2882.4±22.0	71224.8±422.5	143.8±8.8 C:10
B7	10	10644.3±96.5	257606.8±2082.0	106.5±6.8 C:10
B8	10	2169.6±113.0	49393.2±3335.9	17.7±4.9 C:10
B9	10	1884.1±25.3	45075.8±729.3	14.3±3.1 C:10
C1	10	80.6±15.5	1987.7±394.1	210.6±8.3 C:10
C2	10	498.9±12.2	12082.0±451.0	217.9±13.8 C:6,X:4
C3	10	5446.3±194.7	130538.5±4511.6	300.7±29.1 X:9,C:1
C4	10	1221.2±48.9	28860.8±1645.7	328.5±22.5 C:7,X:3
C5	10	3896.1±25.6	94880.1±928.3	150.9±9.5 C:10
C6	10	3075.7±34.4	73511.0±562.9	166.8±9.3 C:10
C7	10	10800.6±129.0	260260.9±2942.4	139.2±9.8 C:9,X:1
C8	10	2585.4±86.2	61776.8±1863.7	22.3±2.2 C:8,S:2
C9	10	2145.0±69.8	53003.8±1613.7	32.1±5.0 C:10
D1	15	71.7±2.3	1780.7±45.2	207.0±17.0 C:15
D2	10	502.7±7.5	12267.7±387.9	226.7±14.0 C:10
D3	12	5428.1±104.0	132539.2±2868.9	305.3±20.3 C:12
D4	10	1223.3±29.7	27863.9±1496.9	321.4±11.1 C:10
D5	10	3914.9±20.9	93659.0±573.1	152.7±9.3 C:10
D6	12	3067.1±29.5	75436.1±712.2	170.1±11.1 C:12
D7	12	10631.2±472.9	254332.1±9492.1	128.2±26.6 C:12
D8	12	2670.2±87.3	63646.7±1977.0	22.8±2.9 C:12
D9	11	2126.7±15.7	51223.2±621.1	28.9±2.7 C:11
E1	10	73.9±4.5	1825.0±107.6	210.9±9.3 C:10
E2	10	503.7±7.1	12416.6±220.9	215.8±7.5 C:9,O:1
E3	10	5469.8±215.7	130564.7±4604.1	307.8±21.3 X:5,C:5
E4	10	1215.4±63.7	27894.3±1737.6	311.6±22.4 X:5,C:5
E5	10	3858.3±32.8	93123.6±924.6	138.6±14.3 C:10
E6	10	3054.2±20.5	72378.5±1049.8	162.1±4.4 C:10
E7	10	10830.0±91.3	259377.2±2649.6	130.9±9.0 C:10
E8	6	2648.2±116.0	62349.5±1726.9	22.5±2.1 C:6
E9	10	2068.1±44.5	50385.1±941.3	27.1±4.4 C:10
E ₉₉ 1	8	71.0±0.0	1743.4±53.5	211.5±13.7 C:8
E ₉₉ 2	20	498.6±7.6	12314.1±296.6	223.6±10.0 C:18,X:2
E ₉₉ 3	20	5467.1±147.2	133069.1±3936.3	307.4±20.6 C:19,X:1
E ₉₉ 4	20	1200.3±31.2	27522.1±1399.2	306.6±18.2 C:20
E ₉₉ 5	20	3846.7±30.9	94251.8±941.2	157.4±12.8 X:3,C:17
E ₉₉ 6	3	3117.0±11.5	74558.9±497.9	171.3±9.1 C:3
E ₉₉ 7	5	10907.6±53.8	265303.7±2194.1	136.2±5.9 C:4,S:1
E ₉₉ 9	12	2069.5±58.0	50428.9±1260.2	27.5±5.3 C:12
E ₁₀₀ 1	4	71.0±0.0	1726.2±74.8	215.2±7.0 C:4
E ₁₀₀ 2	10	500.3±5.7	12417.2±192.6	223.1±10.6 C:9,X:1
E ₁₀₀ 3	10	5429.7±158.7	131751.6±3888.9	304.4±22.5 X:2,C:8
E ₁₀₀ 4	10	1181.2±23.6	26741.3±1359.4	302.8±16.7 X:1,C:9
E ₁₀₀ 5	10	3837.1±33.1	93785.6±875.9	155.9±12.6 C:10
E ₁₀₀ 6	2	3110.5±3.5	74277.3±141.8	169.5±12.0 C:2
E ₁₀₀ 7	3	10909.7±72.5	264791.4±2646.5	135.7±8.1 C:3
E ₁₀₀ 9	6	2065.7±80.9	50258.1±1724.9	26.5±3.1 C:6

References

- [1] V.-T. Pham, M. Böhme, A. Roychoudhury, AFLNet: A greybox fuzzer for network protocols, in: 2020 IEEE 13th International Conference on Software Testing, Verification and Validation (ICST), 2020, pp. 460–465. doi:10.1109/ICST46399.2020.00062.
- [2] R. Natella, StateAFL: Greybox fuzzing for stateful network servers, *Empirical Software Engineering* 27 (2022). doi:10.1007/s10664-022-10233-3, art. no. 191.
- [3] R. Meng, M. Mirchev, M. Böhme, A. Roychoudhury, Large language model guided protocol fuzzing, in: 31st Annual Network and Distributed System Security Symposium (NDSS 2024), 2024, pp. 1–17. doi:10.14722/ndss.2024.24556.
- [4] S. Qin, F. Hu, Z. Ma, B. Zhao, T. Yin, C. Zhang, NSFuzz: Towards efficient and state-aware network service fuzzing, *ACM Transactions on Software Engineering and Methodology* 32 (2023) 1–26. doi:10.1145/3580598.
- [5] J. Ba, M. Böhme, Z. Mirzamomen, A. Roychoudhury, Stateful greybox fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3255–3272. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/ba>.
- [6] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, L. Zhang, Fuzz4All: Universal fuzzing with large language models, in: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13. doi:10.1145/3597503.3639121.
- [7] J. Wang, L. Yu, X. Luo, LLMIF: Augmented large language model for fuzzing IoT devices, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 881–896. doi:10.1109/sp54263.2024.00211.
- [8] P. Srivastava, M. Payer, Gramatron: effective grammar-aware fuzzing, in: *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 244–256. doi:10.1145/3460319.3464814.
- [9] D. Wang, Y. Li, Z. Zhang, K. Chen, CarpetFuzz: Automatic program option constraint extraction from documentation for fuzzing, in: 32nd USENIX Security Symposium (USENIX Security 2023), 2023, pp. 1919–1936. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/wang-dawei>.
- [10] A. Borchherding, M. Giraud, I. Fitzgerald, J. Beyerer, The bandit’s states: Modeling state selection for stateful network fuzzing as multi-armed bandit problem, in: 2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), 2023, pp. 345–350. doi:10.1109/EuroSPW59978.2023.00043.
- [11] W. Zhang, J. Chen, S. Cai, K. Wang, Y. Liu, H. Ding, A novel seed scheduling scheme using Thompson sampling for coverage-guided greybox fuzzing, *Journal of Systems and Software* 236 (2026) 112794. doi:10.1016/j.jss.2026.112794.
- [12] K. Jian, Y. Zou, C. Wang, N. Li, M. Li, W. Huo, State significance-guided fuzzing for stateful protocol program, in: P. Rümmer, Z. Wu (Eds.), *Theoretical Aspects of Software Engineering*, volume 15841 of *Lecture Notes in Computer Science*, Springer Nature Switzerland, Cham, 2026, pp. 361–379. doi:10.1007/978-3-031-98208-8_21.
- [13] R. Natella, V.-T. Pham, ProFuzzBench: A benchmark for stateful protocol fuzzing, in: *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 662–665. doi:10.1145/3460319.3469077.
- [14] M. Schloegel, N. Bars, N. Schiller, L. Bernhard, T. Scharnowski, A. Crump, A. Ale-Ebrahim, N. Bissantz, M. Muench, T. Holz, SoK: Prudent evaluation practices for fuzzing, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 1974–1993. doi:10.1109/sp54263.2024.00137.
- [15] M. Böhme, L. Szekeres, J. Metzner, On the reliability of coverage-based fuzzer benchmarking, in: *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1621–1633. doi:10.1145/3510003.3510230.
- [16] A. Hazimeh, A. Herrera, M. Payer, Magma: A ground-truth fuzzing benchmark, *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4 (2020) 1–29. doi:10.1145/3428334.
- [17] W. R. Thompson, On the likelihood that one unknown

- probability exceeds another in view of the evidence of two samples, *Biometrika* 25 (1933) 285–294. doi:10.1093/biomet/25.3-4.285.
- [18] G. W. Brier, Verification of forecasts expressed in terms of probability, *Monthly Weather Review* 78 (1950) 1–3. doi:10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.
- [19] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, PMLR, 2017, pp. 1321–1330. URL: <https://proceedings.mlr.press/v70/guo17a.html>.
- [20] J. Davis, M. Goadrich, The relationship between Precision-Recall and ROC curves, in: *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 233–240. doi:10.1145/1143844.1143874.
- [21] B. Efron, Bootstrap methods: Another look at the jackknife, *The Annals of Statistics* 7 (1979). doi:10.1214/aos/1176344552.
- [22] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, *The Annals of Mathematical Statistics* 18 (1947) 50–60. doi:10.1214/aoms/1177730491.
- [23] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions, *Psychological Bulletin* 114 (1993) 494–509. doi:10.1037/0033-2909.114.3.494.
- [24] Y. Benjamini, Y. Hochberg, Controlling the false discovery rate: A practical and powerful approach to multiple testing, *Journal of the Royal Statistical Society Series B: Statistical Methodology* 57 (1995) 289–300. doi:10.1111/j.2517-6161.1995.tb02031.x.
- [25] E. L. Kaplan, P. Meier, Nonparametric estimation from incomplete observations, *Journal of the American Statistical Association* 53 (1958) 457–481. doi:10.1080/01621459.1958.10501452.